SERVICE-ORIENTED ARCHITECTURE (SOA) PROJECT DELIVERY SUPPLEMENTAL GUIDE

Back to View

The materials presented in this technical guideline provide specific supplemental information into OUM for the purposes of SOA project delivery work. The Service-Oriented Architecture (SOA) Project Delivery Supplemental Guide contains the following components:

SOA Project Delivery Overview

This component provides an overview of the concepts of SOA Project Delivery.

SOA Project Delivery Supplemental Task Guidelines

The Supplemental Task Guidelines should be used in conjunction with the standard OUM tasks in order to accomplish SOA project delivery work at the detail task level.

SOA Project Delivery Service Interoperability Guidelines

The Service Interoperability Guidelines provide guidelines, patterns and templates to capture all the necessary information related to service interoperability.

SOA Application Integration Architecture (AIA) Supplemental Guide

The SOA Application Integration Architecture (AIA) Supplemental Guide provides an overview of Application Integration Architecture (AIA) use for Application Integration solutions in the context of SOA.

SOA Links to Other Information

There are technical guidelines links for SOA Implementation projects.

PDF View

You may also prefer to access this information through the **Service-Oriented Architecture Project Delivery Supplemental Guide (PDF View)** located in the Supplemental Guides section of the Supplemental Guidance page.

Back to Top

PURPOSE

The purpose of this document is to provide an overview of SOA implementation either as a stand-alone project, or in conjunction with projects using SOA to deliver composite application solutions, as well as process integration. The intent is to provide Oracle implementation teams with *supplemental guidance* for the successful planning and execution of SOA projects.

Implementing a Service-Oriented Architecture (SOA) approach and implementing Enterprise Integration are highly complementary efforts.

SOA concerns itself with improving business agility through reuse of modular business services for both new and existing applications. Use of a common standard (typically web services) for such services promotes interoperability. Interoperability is at the core of Enterprise Integration. As such, SOA can be seen as easing the means of Integration.

Enterprise Integration preceded the evolution of SOA. Historically, it began as a means to better leverage existing IT investments through simple interoperability of disparate applications. Currently, integration has grown to also deal with important issues of data reconciliation, process orchestration, process monitoring, exception management and event processing. As such, Enterprise Integration can be seen as fulfilling a critical role in enriching SOA.

SOA-based Integration, then, can provide an effective means to optimizing new and existing business processes. SOA ROI can often be most quickly realized by reducing the “drag” on business agility caused by brittle point-to-point, hard-coded integration. In today’s heterogeneous IT environment, engagements involving the development of custom developed software solutions or installation of commercial off the shelf (COTS) applications typically involve the integration of third party applications and data sources. Such integration can include batch data loading, real-time data synchronization, trading partner communications, application integration, and business process orchestration.

Once existing applications are integrated as SOA services, they are amenable to enhancement, reconfiguration, enrichment and evolution through monitoring and closed loop feedback, providing the means for business process optimization.

This document helps practitioners use OUM to achieve project goals in that context.

Back to Top

AUDIENCE

This document is intended for practitioners who will be designing and implementing SOA solutions based on the OUM (Oracle Unified Method) approach. Typically, all integration projects using Oracle Fusion middleware will tend to fall into this category, whether the customer is explicitly requesting a SOA approach or not.

Back to Top

HOW TO USE THIS DOCUMENT

This document contains supplemental guidance for planning and executing SOA implementations and should be used in conjunction with and not instead of the OUM base method content.

Back to Top

LIMITATIONS TO THIS DOCUMENT

This document does not describe OUM in detail – the reader must review the documentation and training materials referenced below to gain a basic understanding of each.

Back to Top

RELATED DOCUMENTATION AND TRAINING

The following related documentation & training provide additional detailed guidance related to OUM implementations and are prerequisites to this document:

- OUM Method Pack – Desktop Installable or Hosted version
- OUM Training and Accreditation Program

Back to Top

CONTRIBUTORS

Marc Adlam, Stephen Anderson, Jeffrey K. Bailey, Shamik Chakraborty, Manas Deb, Nishi Deokule, Arunkumar Ganapathy, Troy Hedges, Jan Kettenis, Aali Masood, Tam Murari, Prasen Palvankar, Rajesh Nigam, Bill Peng, Larry Parks, Eric Sahm, Jeffrey Smith, Matt Sorgenfrei, Lynn Lusk, Rijn Van Lynden, Sherwood Zern, Vidya Venkat, Vincent Yin, Stephen Verba

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

SERVICE-ORIENTED ARCHITECTURE (SOA) PROJECT DELIVERY OVERVIEW

Back to View

INTRODUCTION

The Service-Oriented Architecture (SOA) Project Delivery Supplemental Guide should be used to help organizations to execute projects within a Service-Oriented Architecture (SOA) environment, either as an initial project, or as subsequent projects. It should help the customer to support SOA execution.

The focus of this guide is on delivery, but it also provides reference to the strategic tasks in the event that the project needs to first plan portions of the customer's SOA strategy at the enterprise level, in order to accomplish delivery of the solution.

SOA projects based on the SOA Project Delivery view may be planned in combination with the delivery of a larger software implementation project, or the view may be used to support a standalone SOA implementation. The SOA Project Delivery View refers to three approaches (Strategic, Pragmatic and Tactical).

OUM also provides a SOA Tactical Project Delivery view. The SOA Tactical Project delivery view is a more agile approach to SOA used during projects where SOA is being employed to solve practical problems. Often things like a services repository, or establishing services reuse at the enterprise level are not the main objective of these projects.

This guide provides SOA specific supplemental guidance for OUM tasks. In practice you will use these tasks in the context of a SOA implementation project delivery.

The SOA Project Delivery Supplemental Guide also provides a link to the SOA Application Integration Architecture (AIA) Supplemental Guide.

Background

Service-Oriented Architecture (SOA) is an Information Technology (IT) strategy that, in many cases, spans the entire enterprise. IT projects are implemented to fit within an SOA and are therefore just a piece of the larger SOA initiative. If projects are developed independently, without consistent tie-in and guidance from the overall SOA leadership team, the effectiveness and value of each delivery is substantially reduced. Fundamental to successful SOA adoption is the establishment of an Enterprise Service Engineering Framework which is used to tie project delivery and operations into the overall SOA Governance structure and Reference Architecture, as well as apply the best usage of investment in Service Infrastructure.

Back to Top

SOA PROJECT DELIVERY APPROACHES

Organizations can approach their use of SOA in several ways:

Strategic SOA Approach – This encompasses both enterprise and project SOA engagements in a unified, top-down effort to bring the benefits of SOA to the entire organization and involves goals for significant SOA maturity growth. From an OUM perspective this means doing the appropriate Envision tasks for the enterprise and then proceeding to SOA projects using Implement tasks. Specifically, tasks for SOA Roadmap Creation and SOA Strategy and Planning precede project implementation tasks. Therefore, a strategic approach to SOA would consider ALL the tasks in the SOA Project Delivery view.

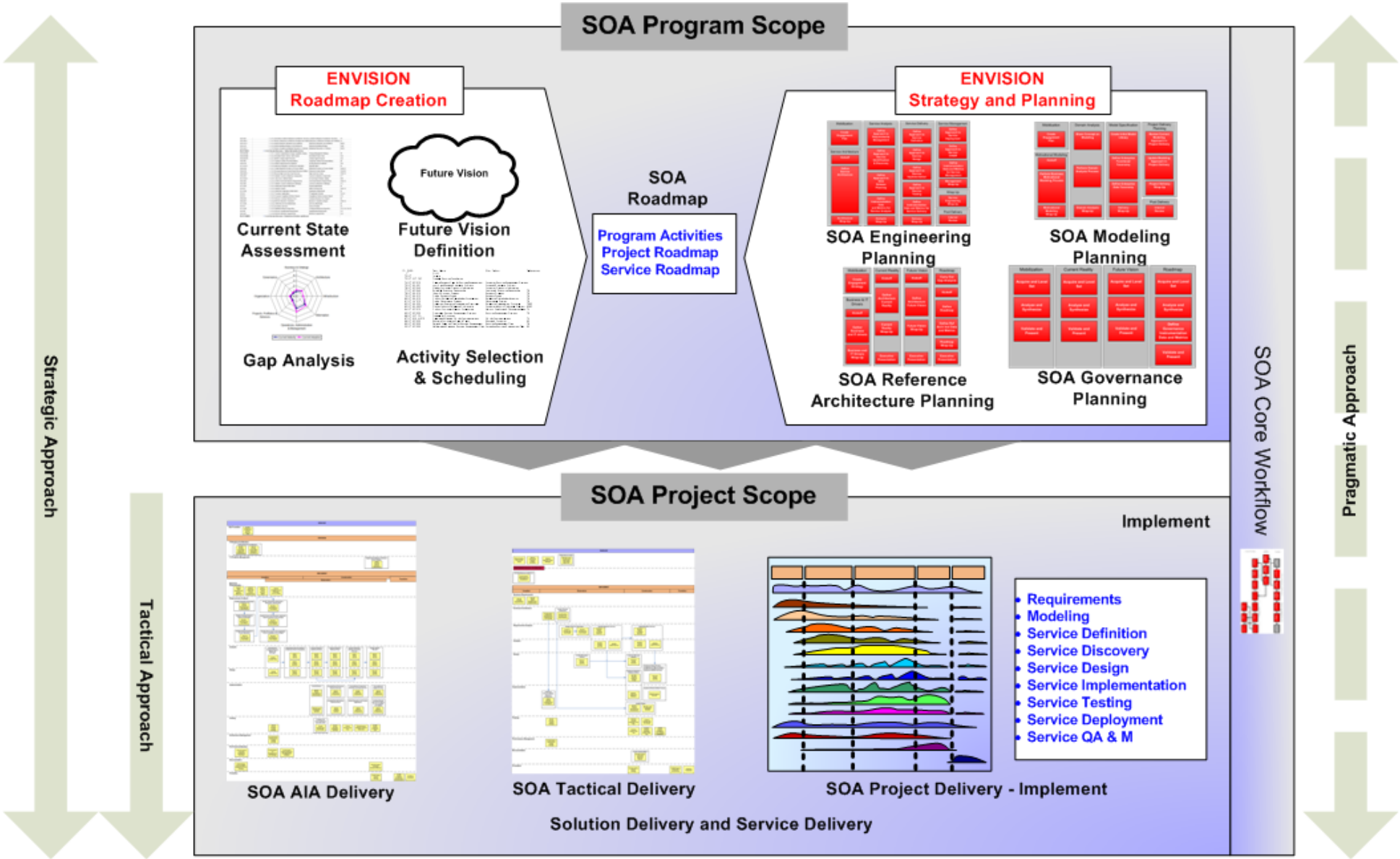
SOA Pragmatic Approach – This occurs when the organization begins their SOA efforts at the project level, but uses this as a springboard to begin to make progress at accomplishing some of the work normally done at the enterprise level in order to begin to gain the larger benefits of SOA. This allows for incremental improvement in SOA maturity. From an OUM perspective, the entry point is at OUM Implement, but with some selected tasks from SOA Strategy and Planning. Therefore, a pragmatic approach to SOA would include Implement focus area tasks and Envision tasks only as deemed necessary by the project.

SOA Tactical Approach – This occurs when SOA is used to solve business problems at the project level without regards to the enterprise-level tasks needed to build full SOA capability. Here the priority is eliminating specific pain points using SOA rather than growth of SOA maturity. From an OUM perspective, a SOA Tactical Approach only involves tasks at the OUM Implement level. This approach does not include SOA Roadmap Creation or SOA Strategy and Planning (Envision) tasks. Therefore, a tactical approach to SOA would ONLY include Implement focus area tasks.

There are three views available for your SOA tactical project, you can use the SOA Project Delivery view and include only those tasks relevant for your project, you can use the SOA Tactical Project Delivery view or the SOA Application Integration Architecture view.

The SOA Tactical Project Delivery view has been pre-tailored for tactical SOA Projects. You may choose to use this as a starting point, and then tailor-up using tasks from the SOA Project Delivery view based on the characteristics of your project.

The Application Integration Architecture (AIA) view is used to execute SOA implementations that utilize Oracle AIA that provides a “pre-built SOA” where significant aspects of a Strategic SOA Approach have been done in advance so that a specific SOA integration project can be completed swiftly and consistently.



Within this context, for example, one could see how a SOA Application-to-Application integration project could be:

- part of a larger Strategic SOA Approach where prior enterprise work was done to establish a SOA Roadmap and SOA Strategy and Planning
- part of a Pragmatic SOA Approach effort that solves the immediate integration problem while also seeking to define some services designed for reuse via limited forays into enterprise-level SOA
- part of a Tactical SOA Approach effort that focuses on resolving the immediate problem

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

SERVICE-ORIENTED ARCHITECTURE (SOA) PROJECT DELIVERY SUPPLEMENTAL TASK GUIDELINES

Back to View

This document contains OUM supplemental task guidance for SOA project delivery.

- The first column of the following table provides a complete list of all tasks in this document for which "supplemental" guidelines are provided as well as a quick link to the "supplemental" task guideline in this document.
- The second column provides a link to the OUM Task Overview.

The task guidelines in this document should be used to supplement the guidelines that appear in the OUM Task Overview.

SOA Project Delivery - OUM Supplemental Task Guidelines (this document) collapse table | expand table	OUM Task Overview
Configuration Management	
CM.040 Create and Execute Software Configuration Management Plan	CM.040 Create and Execute Software Configuration Management Plan
CM.050 Create and Execute Software Release Management Plan	CM.050 Create and Execute Software Release Management Plan
Envision Roadmap	
ER.130 Produce MoSCoW Improvement List	ER.130 Produce MoSCoW Improvement List
Enterprise Business Analysis	
BA.012 Define Business Scope	BA.012 Define Business Scope
BA.040 Create Enterprise Function or Process Model	BA.040 Create Enterprise Function or Process Model
BA.050 Develop Enterprise Domain Model (Business Entities)	BA.050 Develop Enterprise Domain Model (Business Entities)
BA.060 Perform High-Level Use Case Discovery	BA.060 Perform High-Level Use Case Discovery
Enterprise Architecture	
EA.010 Review Architecture Principles, Guidelines and Standards	EA.010 Review Architecture Principles, Guidelines and Standards

EA.075 Develop Technology Reference Architectures	EA.075 Develop Technology Reference Architectures
IT Portfolio Management	
IP.080 Maintain Repository of Artifacts	IP.080 Maintain Repository of Artifacts
Business Requirements	
RD.011 Develop Future Process Model	RD.011 Develop Future Process Model
RD.020 Obtain High-Level Business Descriptions	RD.020 Obtain High-Level Business Descriptions
RD.045 Prioritize Requirements (MoSCoW)	RD.045 Prioritize Requirements (MoSCoW)
RD.055 Detail Supplemental Requirements	RD.055 Detail Supplemental Requirements
RD.065 Develop Domain Model (Business Entities)	RD.065 Develop Domain Model
Requirements Analysis	
RA.019 Define Project Reference Architecture	RA.019 Define Project Reference Architecture
RA.023 Develop Use Case Model	RA.023 Develop Use Case Model
RA.024 Develop Use Case Details	RA.024 Develop Use Case Details
RA.025 Identify Candidate Services	RA.025 Identify Candidate Services
RA.026 Populate Services Repository	RA.026 Populate Services Repository
RA.085 Validate Functional Prototype	RA.085 Validate Functional Prototype
Design	
DS.040 Develop Design Architecture Description	DS.040 Develop Design Architecture Description
DS.070 Define SOA Implementation Strategy	DS.070 Define SOA Implementation Strategy
Implementation	
IM.010 Develop Functional Prototype	IM.010 Develop Functional Prototype
IM.050 Implement Components	IM.050 Implement Components
IM.060 Perform Component Review	IM.060 Perform Component Review
Testing	
TE.010 Develop Testing Strategy	TE.010 Develop Testing Strategy
TE.015 Develop Integration Test Plan	TE.015 Develop Integration Test Plan
TE.020 Develop Unit Test Scripts	TE.020 Develop Unit Test Scripts
TE.030 Perform Unit Tests	TE.030 Perform Unit Tests
TE.040 Perform Integration Test	TE.040 Perform Integration Test
Technical Architecture	
TA.020 Define Technical Architecture Requirements and Strategy	TA.020 Define Technical Architecture Requirements and Strategy
TA.030 Define Integration Requirements and Strategy	TA.030 Define Integration Requirements and Strategy

TA.090 Develop Security and Control Strategy	TA.090 Develop Security and Control Strategy
Data Acquisition and Conversion	
CV.020 Define Data Acquisition, Conversion and Data Quality Strategy	CV.020 Define Data Acquisition, Conversion and Data Quality Strategy
CV.030 Prepare Conversion Environment (Initial Load)	CV.030 Prepare Conversion Environment (Initial Load)
CV.040 Design Conversion Components (Initial Load)	CV.040 Design Conversion Components (Initial Load)
CV.055 Implement Conversion Components (Initial Load)	CV.055 Implement Conversion Components (Initial Load)
CV.060 Perform Conversion Component Unit Test (Initial Load)	CV.060 Perform Conversion Component Unit Test (Initial Load)

Back to Top

CONFIGURATION MANAGEMENT (CM) PROCESS GUIDANCE

CM.040 Create and Execute the Software Configuration Management Plan

As part of the Portfolio Management Strategy for SOA, you should define the version management strategy for the relevant assets, and thereafter synchronize change management with this strategy. Review the section “Service Versioning Strategy” in the Service Lifecycle technique for more information. In initial SOA implementations there may not be an Enterprise strategy around Portfolio Management. In that situation you will need to define enough structure to support the implementation, and encourage the enterprise to develop the Portfolio Management Strategy in conjunction with the initial implementation of SOA within the Enterprise.

CM.050 Create and Execute the Software Release Management Plan

A software release refers to the creation and availability of a new version of computer software product or a group of configuration items.

Release planning is the activity of determining a feasible combination of dates, features, components and resourcing for the next release of an existing software product.

PREREQUISITES

There are many prerequisites for this task according to the methodology. Unfortunately, the customer may not have all of the desired prerequisites and in some instances, they may not necessary if this is going to be a pure integration delivery. However, for both the Pragmatic and Tactical SOA approaches, the Configuration Management Plan and Processes (CM.010) is a required prerequisite.

Back to Top

ENVISION ROADMAP (ER) PROCESS GUIDANCE

ER.130 Produce MoSCoW Improvement List

When a new Enterprise Requirement has been identified, either as a request for reuse of an existing asset, or as an extension to an existing asset, or as a complete new asset, this needs to be prioritized. Each requirement will have an assigned ownership. For further details about this, review the Enterprise Requirements Management technique.

Back to Top

ENTERPRISE BUSINESS ANALYSIS (BA) PROCESS GUIDANCE

BA.012 Define Business Scope

SOA requirements management is somewhat different from traditional requirements management. In a traditional approach, requirements could be managed in project scope, i.e. each project is defining and managing its own requirements to create a solution in the scope of the project. But in SOA the goal is to share services and as a consequence projects will need to start sharing the corresponding requirements as well. Therefore, it is important to manage the requirements related to services at enterprise level. Depending on the service engineering approach, this may also lead to the need to manage all requirements at the enterprise level, to make requirements visible across projects hence supporting the service identification and discovery approach. Scoping requirements at the enterprise level, rather than at the application or project level enables project architecture and planning exercises to remain connected to the overall enterprise SOA.

The Business Scope indicates the domain within which the Requirements Management Process should be defined and executed for SOA. This gives the boundaries for the work that should be performed using tasks from both Envision and Implement focus areas.

BA.040 Create Enterprise Function or Business Process Model

The Enterprise Function or Business Process Models should be created or updated whenever required based on new or updated models developed as part of a project. The project models should be classified following the taxonomy of the function or business process model (see the Functional or Process Modeling technique). It will then be easier to identify any possible duplicates or overlaps in the Enterprise Repository.

When the project-level models duplicate models in the Enterprise Repository, you should further examine the project requirements to see whether there is a 100% fit with the Enterprise requirements, or whether the fit is only partial. If there is a 100% fit, this should be marked for potential reuse. If the fit is only partial, you should consider whether it would be feasible to extend the current function with the new requirements. If there is no fit, the new requirements should be registered in the Enterprise Repository for possible later reuse.

For further details about this, review the Enterprise Requirements Management technique.

During Service Identification and Discovery, the Enterprise Function or Business Process Model may be used as a basis for Functional Activity Analysis. Review the Service Identification and Discovery technique for more information about this.

BA.050 Develop Enterprise Domain Model (Business Entities)

The Enterprise Domain Model should be created or updated whenever that is required based on new models developed in a project. The project models should be classified following the taxonomy of the domain model. It will then be easier to identify any possible duplicates or overlaps in the Enterprise Repository.

When there are duplicates in the models, you should look further into the documented requirements to verify whether there is a 100% fit, or whether the fit is only partial. If there is a 100% fit, this should be marked as a potential reuse. If it is only partial, consider whether it would be feasible to extend the current function with the new requirements. If there is no fit, the new requirements should be registered in the Enterprise Repository for possible later reuse.

For further details about this, review the Enterprise Requirements Management technique.

During Service Identification and Discovery, the Enterprise Domain Model is used as a basis for Data Model Analysis. Review the Service Identification and Discovery technique for more information about this.

BA.060 Perform High-Level Use Case Discovery

The High-Level Use Cases should be created or updated whenever that is required based on use cases being identified during a project. The use cases should be classified following the taxonomy of the Use Case Model. It will then be easier to identify any possible duplicates or overlaps in the Enterprise Repository.

When there are duplicates in the models, you should look further into the documented requirements to verify whether there is a 100% fit, or whether the fit is only partial. If there is a 100% fit, this should be marked as a potential reuse. If it is only partial, consider whether it would be feasible to extend the current function with the new requirements. If there is no fit, the new requirements should be registered in the Enterprise Repository for possible later reuse.

For further details about this, review the Enterprise Requirements Management technique.

During Service Identification and Discovery, the High-Level Uses Cases are used as a basis for Functional Activity Analysis. Review the Service Identification and Discovery technique for more information about this.

Back to Top

ENTERPRISE ARCHITECTURE (EA) PROCESS GUIDANCE

EA.010 Review Architecture Principles, Guidelines and Standards

You should define all the architectural standards and polices you will need for service engineering. This includes:

- Define a strategy to identify the right service granularity. Review the Service Boundary Analysis technique for details about this.
- Define a service scoring schema. Review the Service Identification and Discovery technique for details about this.
- Define architectural policies. Review the Service Architecture technique for details about this.

EA.075 Develop Technology Reference Architectures

When determining the Technology Reference Architecture for SOA – called the SOA Reference Architecture – there are certain elements that should be covered. For any situation that involves a set of SOA services of any significant size, or if the enterprise has embraced SOA as an IT strategy, it is important to determine and document a consistent strategy for service implementation. Failure to establish such a strategy might result in an inconsistent service implementation and may negate the advantages of using a SOA approach by greatly hampering reuse.

Therefore, during this task, you define the SOA Reference Architecture that will be used at the enterprise level, one of the most important parts being the definition of the layers of the SOA architecture. The SOA Reference Architecture should describe:

- how services should be implemented in the enterprise architecture
- what kind of services should be located where (in what layer or tier)
- what access and communication mechanisms should be used
- a target schedule for implementation over a several year period

The SOA Reference Architecture should cover:

1. Service Definition - What is a service, service scope, service lifecycle, etc.?
2. Conceptual View - service types and layers (for example, connectivity, data, decision, business, business process, presentation services and associated rules and guidelines for each layer or service type)
3. SOA Infrastructure - covers components (for example, a service bus) to perform service routing, service orchestration, governance, management, etc.
4. SOA Security - includes service authorization, identity federation, etc.
5. Implementation - tools and technology for implementing the previous points
6. Deployment - covers service packaging and a service deployment strategy
7. Patterns and Design Guidelines - includes best practices and design patterns

Guidelines for points 1 and 2 above are available in the Service Architecture technique.

Service Definition

Most people have a different understanding of what is a service, depending of their background, level of experience with SOA, etc. The first part of the SOA Reference Architecture is concerned with defining what is a service, what is it's composition, etc. See the Service Architecture technique for more guidance.

Service Layers

SOA layers provide a means to organize application user interfaces and services in similar groupings. There is not necessarily a direct relationship between SOA service layers and the physical infrastructure tiers. SOA service layering is chiefly concerned with establishing the architectural decoupling such that processes and data can be more readily distributed and deployed. It allows for defining different types of services and then the different policies or rules that apply to them. For example, you may define that a data service must use objects from the enterprise canonical data model, whereas a presentation service does not.

See the Service Architecture technique for more details on how to do this.

One aspect in particular that needs to be considered while defining possible SOA service layers is their granularity. The following three aspects are the most important ones that can help you in determining the right granularity:

Business Suitability

Ideally a service supports an atomic operation of a business task. It provides just enough functionality needed for that task. Whenever making it simpler makes it more general, this will increase the level of reusability of the service.

One service that handles a customer with addresses as well as all the orders the customer can place is likely too big. You probably cannot reuse it for some other business task that only involves handling a customer and no order lines (for example, for security reasons).

Performance and Size

Services are called remotely and therefore require a roundtrip, and so a performance overhead is incurred. You therefore want to reduce the calls to a service as much as possible, as long as that does result in services having too many responsibilities (see previous point). For example, rather than identifying a service that handles one order as well as another service that handles one order line, you identify one service that handles an order together with its order lines.

Furthermore, services are limited in the size of the messages they can process efficiently. When a message becomes too big, it will require too many resources to process. Transactions and State operations should be self-contained as much as , to reduce the risk that the state gets lost when a service fails to complete. When a service stores data this should be done in exactly one transaction, and failure of a service should not result in corrupting the data. In other words: between two calls the data will always be in a valid state.

Integration Considerations

During this analysis, try and look at the number of partner links to maximize the business logic to have services that are self-contained. Try designing components that do not have a fine granularity.

There are several ways to approach the integration analysis for defining the integrations and the canonicals. One approach is to think of a hub and spoke model where there is a single canonical that can be used for the services. Another approach is to have a unique canonical for each service. A finally approach may be to utilize an ESB (Enterprise Service Bus) architecture.

Service Infrastructure

The details of the infrastructure required to run the SOA is also an important part of the SOA Reference Architecture. This may include a choice of Enterprise Service Bus, Enterprise Repository, BPM tooling, etc. In this section, describe the functions that the components you need carry out and how these components interact with each

other. It is recommended to leave this section technology and product agnostic and detail the products in the Implementation section below.

SOA Security Strategy

The SOA Security Strategy should include the solutions for the following:

- Service authorization: including how the security is handled at the service level, how authorization policies are managed (locally or centrally)
- Identity propagation: how user identity is passed from one service down to the next
- Service Provisioning: who is allowed to use which services (users or other type of resources in terms of roles), how these services should be used and accessed, and how authorization should be granted

This section should also include a description of how the provisioning process should be monitored, to support governance on the security by the architecture. This aspect should also cover how alerts/reports should be created depending on how services are used (such as halts, interruptions, decline and so on). This should be documented both from a service provider and a service consumer point of view.

Implementation

The "Implementation View" of the SOA Reference Architecture describes the technology and products required to implement the SOA Reference Architecture defined so far.

Deployment Options

Services may be packaged individually or as a group, independently of each other or as part of a common release cycle. In this step, define which strategy the enterprise adopts including the criteria for grouping services, release cycles, etc.
See the SOA Release Planning technique for more details on how to perform this step.

Patterns and Design Guidelines

General **SOA service design principles**:

- **Design for Reuse** - The practitioner should determine the appropriate level of granularity. Design pattern should be employed during the service design. Always pay close attention to interoperability in a heterogeneous IT environment. Make sure your design will survive the disparate and heterogeneous environment.
- **Design for Agility** - The practitioner should consider service virtualization and transparency. Make sure the implementation can be done independently. Try to make your design flexible and extensible by using integration design patterns.
- **Standard for Compliancy** - Make your design standard compliant, where appropriate. Prevent over-standardizing, as (among others) that might jeopardize agility.
- **Database Design** - There should also be design standards for database tables. Oftentimes, this is not required because integration consultants deal mainly with existing data stores and data models.
- **Security** - Incorporate a security model into your design from the start. Always think about security and performance during your design.
- **Performance** - High availability and performance will be critical to the success of any SOA project.
- Transactions - Transaction management is critical to B2B integration, for example.
- **Quality of Service** - QoS is a critical element of any SOA undertaking.
- **Service Interoperability** - In order to provide or consume a ‘Business Service’, the involved systems/partners must agree on the interaction pattern with the corresponding business semantics along with the message binding contracts and policies such as enveloping, addressing, reliability and security profiles for every business service and action pair. The SOA Project Delivery Service Interoperability Guidelines provide guidelines, patterns and templates to capture all the necessary information related to service interoperability.
- **Reporting/Monitoring** - Consider adding BAM sensors or using BPEL "naive table."

Use of **Design Patterns** should be followed whenever possible. Some of the typical integration design patterns include:

- **Canonicals** - Use canonical (common) formats for data representation. Terms associated with canonicals are Common Information Model, Common Object Model, and (standardized) Business Object Definitions. If a Data Hub is being used this may also impose a canonical form that will have to be supported (or mapped to) as well.

- **Content Based Routing** - Use content based routing to map canonical data types to multiple endpoint data types.
- **Process Orchestration** - Use the BPEL engine and workflow engine to handle process orchestration, manual intervention, and fault handling and notification. The BPEL engine acts as a service broker that orchestrates services.
- **ESB** - Use ESB to transform and enrich data and route data to endpoints.

Naming standards should be defined for all project components. The naming standards should be tailored to the products used within the solution (i.e., BPEL PM, B2B). At a minimum, the focus of naming standards should cover:

- service operation names
- event types
- BPEL process names
- BPEL activity names
- Java packages

At a minimum, **coding standards** should address:

- Common WSDL design standards such as standard fault replies for synchronous web services/business processes
- Common error handling approach for BPEL processes
- Common logging approach
- How to define the standard canonicals

Contract/Interface First - Define the interface first. Interfaces determine the contract between service consumers and providers.

- **Coarse Grain Documents** - Provide document on coarse-grained service.
- **Asynchronous Interactions** - Pay special attention to asynchronous interactions between service consumer and provider.
- **Undo/Cancel Operations** - Create a strategy to support transactional services. Determine which services should have the undo/cancel/roll back operation. Determine how you should implement the rollback service.
- **Version Control** - Establish a version control mechanism for services, preferably under the guidance of SOA Governance policies for this purpose. Services will evolve and a version control process and mechanism for implementing and monitoring versioning will be important for longer term enterprise SOA efforts.
- **WS-I, Wrapped Document Style** - Pay attention to Web-Service Interoperability (WS-I) if your solution involves inter-enterprise service orchestration. WS-I creates, promotes and supports generic protocols for the interoperable exchange of messages between Web services. In this context, "generic protocols" are protocols that are independent of any action indicated by a message, other than those actions necessary for its secure, reliable and efficient delivery, and "interoperable" means suitable for multiple operating systems and multiple programming languages.
- **WSIF Binding to Java, JCA** - Consider WSIF for Java binding and integrating with endpoint through JCA.

Connectivity and messaging mechanisms:

- **Consider the use of UDDI registry for services** - Oracle has a UDDI registry product bundled with the application server.
- **JCA Adapters** - JCA adapter is extensively used for integration with backend systems. Practitioners should understand the mechanism of JCA and ways to implement integrations using JCA adapter.
- **Service Virtualization** - Virtual services are the exposed interfaces that a consumer connects to in Web Services. It is not the "real" implemented service interface. Rather, it is a service contract that has a different set of attributes defining the service interface and provides a contract to which the service consumer can connect. It is an abstraction layer between the service consumer and the actual service interface. Whenever the service interface or address needs to be changed, the service consumer will not be affected because the consumer is interacting with the virtual service interface. Another important characteristic of service virtualization is that it creates a policy enforcement layer where SOA governance can be implemented and executed. With the virtual service and supporting abstract layer, services, trust domains, and platform implementations can be chosen at run time according to policies.

Process Orchestration, Workflows and Rules:

- **BPEL** - BPEL PM is the ideal language for designing and deployment of service orchestration.
- **XSLT Transformation** - XSLT is the must-have skill for designing and implementing BPEL process.
- **Human Workflow Service** - In many cases, human intervention is required in a business flow. BPRL PM provides out-of-the-box workflow services for

customization. Basic understanding of human workflow concepts and Java servlet, JSP programming is essential for successful customization of a human workflow.

- **Consider implementing Rules** - Rules-based "Decision Support Service" for complex transformations via Oracle Rules Engine.
- **Notification Service** - You have to have a notification service for critical event notification.
- **Error Hospital Service** - The Error handling framework should be in place to take care of any errors.
- **ESB Binding and Wiring** - Consider using ESB for connectivity and content based routing.
- **Tracing and Debugging** - BPEL provides a nice tool for tracing and logging.
- **Iterative Development** - Always follow the iterative development method.
- **Unit Testing** - Integration development should have unit testing.
- Business Rules Engine - SOA-based design recognizes that services deal with differing degrees of rule logic complexity. In particular, it is considered a best practice to separate process flow rules from complex transformation or calculation rules. With the emergence of Business Rules engine technology, Integration/Enterprise Architects should consider a Business Rules Engine as part of the solution when such complex transformations are needed. Best practice calls for delegation to a Decision Support Service implemented via rules engines that encapsulates business rules creation, maintenance, and management for large rules sets to a centralized repository. The Business Process Developer should be able to leverage the rule engine resource to create highly maintainable processes with simple and easy to maintain flows. You should review the OUM tasks that are specific to business rules:

Task ID	Task Name	Work Product Name	Phase
Envision			
BA.080	Identify Candidate Enterprise Business Rules	Candidate Business Rules	Initiate
IP.030	Analyze Business Rules	Business Rules Analysis	Initiate
EA.160	Define Business Rules Implementation Strategy	Business Rules Implementation Strategy	Initiate
BA.110	Maintain Business Rules	Maintained Business Rules	Maintain and Evolve
IP.080	Maintain Repository of Artifacts (for Business Rules)	Maintained Repository of Artifacts (for Business Rules)	Maintain and Evolve
Implement			
RA.027.1	Identify Candidate Business Rules	Candidate Business Rules	Inception
RA.028.1	Populate Business Rules Repository	Populated Business Rules Repository	Inception
RA.027.2	Identify Candidate Business Rules	Candidate Business Rules	Elaboration
RA.028.2	Populate Business Rules Repository	Populated Business Rules Repository	Elaboration
AN.070.1	Analyze Business Rules	Business Rules Analysis	Elaboration
DS.060	Define Business Rules Implementation Strategy	Business Rules Implementation Strategy	Elaboration
DS.110.1	Design Business Rules	Business Rules Design	Elaboration
IM.055.1	Perform Business Rules Implementation (Rules Engine)	Implemented Business Rules (Rules Engine)	Elaboration
RA.027.3	Identify Candidate Business Rules	Candidate Business Rules	Construction
RA.028.3	Populate Business Rules Repository	Populated Business Rules Repository	Construction
AN.070.2	Analyze Business Rules	Business Rules Analysis	Construction
DS.110.2	Design Business Rules	Business Rules Design	Construction
IM.055.2	Perform Business Rules Implementation (Rules Engine)	Implemented Business Rules (Rules Engine)	Construction

The SOA Reference Architecture template can be used to document this task.

Back to Top

IT PORTFOLIO MANAGEMENT (IP) PROCESS GUIDANCE

IP.080 Maintain Repository of Artifacts

Service Engineering will, in addition to the service asset itself, require other assets along the service lifecycle, such as models. If there are no existing meta models for those other assets, or if the meta models do not support SOA, then you will need to create or update these meta models as part of this task. You need to do this unless there is another initiative to define meta models for these artifacts. Either way, you need to be certain that the meta models support what is needed by SOA.

When a meta-model update is required, you will identify or define the taxonomy that should be used for all artifacts that are related to a SOA, with exception of the taxonomies for Projects, Applications and Services, as these are defined in their respective tasks, Determine Projects Meta Data Description (GV.092), Determine Applications Meta Data Description (GV.094) and Determine Services Meta Data Description (GV.096). You thereby perform the step “Determine Asset Type Taxonomies”. Review the IT Asset Management technique to perform this task.

There may already be an Enterprise Repository available, or the organization has only decided which Enterprise Repository should be used, but it is not yet available, or ther eis no Enterprise Repository. The Enterprise Repository is needed to store and maintain all the required types of artifacts. For SOA, the Enterprise Repository should store all the service definitions, as well as the artifacts that are needed along the lifecycle of the service. The actual implementation of the assets in the Enterprise Repository is performed as part of the task, Implement Governance (GV.100).

For the type of assets you need for SOA, in addition to determining the taxonomies, you should perform these steps these materials are not already available :

- Determine asset type lifecycle
  to determine the lifecycle of the assets related to SOA, with the exception of Projects, Applications and Service as that is performed in their respective tasks (GV.092, GV.094 and GV.096 as mentioned above)
- For these assets, determine asset type properties
  to determine how the assets should be documented in relation to SOA
- Determine assets relationships
  to determine how these assets are required throughout the identified Service Lifecycle.

All of this work is all required to define a proper Portfolio Management Strategy for SOA. Review the IT Asset Management technique for more information on this.

Back to Top

BUSINESS REQUIREMENTS (RD) PROCESS GUIDANCE

RD.011 Develop Future Process Model

When the Future Process Model has been developed during a project, the model should be verified against the Enterprise Repository for reuse. You should verify this as early as possible, after having defined only the highest levels of the process model, to see whether you can identify any duplicates or partial duplicates that you may reuse in your modeling efforts. This verification should be done multiple times during the process, as you might discover new aspects along the way, which may result in new mappings. You should classify your future process model following the process model taxonomy. That makes it easier to identify any duplicates.

Review the Enterprise Requirements Management technique for further information about this.

During Service Identification and Discovery, the Future Process Model may be used as a basis for Functional Activity Analysis. Review the Service Identification and Discovery technique for more information about this.

In this task, you define the future business model in the form of integrated process flows based on the business processes supported by the new applications.

PREREQUISITES

There are many prerequisites for this task according to the methodology. Unfortunately, the customer may not have all of the desired prerequisites and in some instances, they may not necessary if this is going to be a pure integration delivery. For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Service Portfolio - Candidate Services (ENV.BA.090)	Tentative	Optional	For the Pragmatic SOA Approach, obtain the enterprise-level documents if they are available.
Process Improvement Options (ENV.BA.090)	Optional	Optional	
Business and System Objectives (RD.001)	Required	Required	The Business and System Objectives are used to determine what should be the content of the new application.
System Context Diagram (RD.005)	Required	Required	The System Context Diagram provides the boarders for the Future Process Model.
Current Process Model (RD.030)	Required	Required	Use material from the Current Process Model to gain an understanding of the processes and systems that support the current business. It may identify processes that offer opportunities for process improvement in cycle time, cost, and quality. It may also show major processes that are candidates for process redesign.
Current Business Baseline Metrics (RD.034)	Optional	Optional	For projects where the focus is on business performance improvement, the Current Business Baseline Metrics is used to identify the critical performance areas that need to be focused on while developing the Future Process Model. Without this baseline, any business performance improvement (or degradation) becomes very subjective.
Business Use Case Model (RA.015)	Required	Required	If the Business Use Case Model is not available, you may need to create it as part of this effort.
Skilled Project Team (TR.050)	Required	Required	The project team members must understand application concepts and capabilities in order to develop process models based on leading practices. This prerequisite helps them understand what process steps are supplied by application-specific functions.
Business Process Reengineering (BPR) Studies	Optional	Optional	Business Process Reengineering Studies are only appropriate if process change is relevant, and earlier business process change tasks were not carried out — most significantly, Detail System and Business Objectives (RD.001) and Obtain High-Level Business Descriptions (RD.020). Process reengineering studies may recommend the complete overhaul of current processes to be supported by the new applications. The output of a BPR study may, in fact, provide most of the content of the Future Process Model.
Future Business Strategy	Required	Required	This is probably part of the requirements; however, if not, It would make sense to understand so that the services could account for future direction.

In a purely tactical approach, it would be best to tackle the delivery of the Future Process Model from the bottom-up. This involves decomposing the application and identifying sources of functionality to service-enable.

After reviewing the functional requirements, take the following approach when attempting to create a business process model:

1. Note the key verbs and associated nouns.
2. Focus on the “what.”
3. Replace application specific logic with business terminology.

For the data requirement decomposition, consider the following guidelines:

1. Review requirements, noting the nouns.
2. Understand relationships between business entities.
3. Convert pronouns into business entities.

4. Identify possible actions on entities by noting verbs and create a CRUD matrix.
5. Construct domain data hierarchy detailing source, authorities, and semantic communities.

RD.020 Obtain High-Level Business Descriptions

In this task, you collect high-level information about how the users conduct their business for those areas under investigation.

For Tactical SOA and Pragmatic SOA, though this is a required task, the following guidelines apply as engaging in a tactical manner may mean that the requirements have already been identified and the workshops have already been completed.

- Obtain the documents produced by the customer's workshops. Keep in mind, that many times these documents are incomplete and are written in the customer's peculiar industry vocabulary.
- Obtain a glossary from the customer.
- Any follow up discussions or clarifications to the requirements document delivered by the customer need to be added as an addendum to the requirements document.
- Obtain the functional and non-functional requirements of the proposed application.

If an enterprise repository exists, then follow the steps outlined in the OUM.

For those situations where no enterprise repository exists, create a "repository." The repository could be as simple as a spreadsheet. Consider the following Information for each requirement:

- Name
- Classification
- Status
  - Proposed
  - Assigned
  - In Progress
  - Implemented
  - Partially Implemented
  - Retired
- Type
  - Shared
  - Not Shared
  - Owner
- Version

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Configured Enterprise Repository (GV.098)	Optional	Optional	If available, use the Configured Enterprise Repository to check for existing process models and add to or update the process models in the repository.
Business and System Objectives (RD.001)	Tentative	Optional	For the Pragmatic SOA Approach, obtain the enterprise-level objectives if available.
Future Process Model (RD.011)	Required	Required	

During the project that is executed in combination with this view, high-level business requirements are collected and used as input to the modeling tasks. When the requirements are gathered, you should check for duplicates in the Enterprise Repository. You need to update the Enterprise Repository with the new requirements. Review the Enterprise Requirements Management technique for more details about this.

RD.045 Prioritize Requirements (MoSCoW)

When the requirements for the project have been identified, you should also know what kind of requirements may be implemented:

- Through reuse of services (services already implemented, or to be developed in other projects or as part of this project),
- As an extension to existing services
- Not as a service

This is important information when prioritizing. A requirement that can be implemented through reuse may get a higher priority than it otherwise would have as it may be considered to be "low hanging fruit." For further details, review the Enterprise Requirements Management technique.

RD.055 Detail Supplemental Requirements

During this task, you identify enterprise-level tasks and policies.

If the Envision task, GV.030, Discover or Define Policies (or its equivalent) has been previously done for SOA at an enterprise level, then these policies should be used here. If GV.030 or its equivalent has not been done, then depending on business needs, SOA Governance Policies should be identified and created for this specific project.

At the enterprise level, architects have to determine the enterprise governance policies for the SOA project. Examples of governance policies include service acquisition policies, service lifecycle policies, and architecture/design policies.

Service acquisition policies provide guidelines and regulations about how a service should be acquired:

- whether to reuse existing application functionality,
- whether to acquire third-party software,
- or whether to custom build a new application to provide the service.

Service lifecycle policies govern the following aspects of a service lifecycle:

- service specification approval, standards policies - answer the question "what, where, how a standard should be used?"
- certification and testing policies
- change management policies
- production support policies
- service publishing policies
- publication (code) change control

Architecture design policies govern the following aspects:

- Service Layering Policies - Why should we use service layering? How do we achieve layered SOA?
- Service Dependency - determines how services depend on each other (independent services, cyclic dependent services, and cycle dependent services)

Service Design Policies cover aspects of what is needed to design services including:

- invocation (Asynchronous and Synchronous?)
- parameter types (RPC, SOAP, etc.)

- exception handling
- transaction behavior

Reporting requirements must address the monitoring of the SOA solution as well as management level reporting needs. Typical reporting requirements fall into the following areas:

- **System Health** - This reporting is for operations and technical support personnel and provides reporting on the overall health of the solution. The focus is on the hardware and software, and provides such statistics as CPU utilization, memory utilization, event processing errors, message volumes, etc. BAM can be a valuable asset to monitoring system health (see the KPI discussion hereafter) and should be used from the start for, at least, this purpose.
- **B2B** - Reporting showing partner communication volumes and errors. Typically, a B2B solution provides such reporting or custom reports are developed from the B2B data store. Sometimes you have to come up with data schema and data conversion strategy, etc.
- **Key Performance Indicators (KPI)** - With the emergence of BAM solutions, reporting on key performance indicators for executive and management level is supported. This should be implemented using a top-down process of identifying the KPI reporting needs and mapping the data requirements to the events being processed by the integration solution. Integration architects should try their best to incorporate BAM into system monitoring and management solutions. BAM monitors business metrics such as KPI and Service-Level Agreements in real-time. BAM enables IT staff to analyze real-time data to identify bottlenecks and exceptions. With BAM, corresponding (automated or manual) actions can be performed to meet the business needs. BAM requirements should be captured in this task.

RD.065 Develop Domain Model

When the Domain Model has been developed during a project, the model should be verified against the Enterprise Repository for reuse. You should verify this as early as possible to see whether you can identify any duplicates or partial duplicates that you may reuse in your modeling efforts. Frequently, the same data types or entities are used in multiple projects. To prevent multiple definitions of the same entity, you should verify this as early as possible. You should perform this type of verification multiple times during the process, as you might discover new aspects along the way, which may result in new mappings. You should classify your domain model following the domain model taxonomy. That makes it easier to identify duplicates.

You should review the Enterprise Requirements Management technique for further information about this.

During Service Identification & Discovery, the Domain Model may be used as a basis for Data Model Analysis. You should review the Service Identification and Discovery technique for more information about this.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Enterprise Domain Model (ENV.BA.050)	Partial	Optional	For the Pragmatic SOA Approach, obtain the enterprise-level documents if they are available.
Business and System Objectives (RD.001)	Required	Required	The Business and System Objectives are used to determine the content of the new application.
High-Level Business Descriptions (RD.020)	Required	Required	The High-Level Business Descriptions identify the required business objects and their relationships.

Consider the following additional guidelines when developing the Domain Model:

1. Leverage the Enterprise Domain Model if it exists, as it relates to the application.
2. Identify the Domain Model for the application(s) to be built or integrated.
3. Use the “repository” (spreadsheet) to model the services that have been chosen.

Back to Top

REQUIREMENTS ANALYSIS (RA) PROCESS GUIDANCE

RA.019 Define Project Reference Architecture

During this task you define the SOA Project Reference Architecture.

During project start up, you should check and get an understanding of the existing SOA strategy and determine specific project implications. This includes checking the SOA Reference Architecture (EA.075) at the enterprise level and understanding the service portfolio management strategy. If a SOA Reference Architecture or a service portfolio management strategy are lacking in the enterprise, you should create them in the context of the project with the view to extend them to the enterprise whenever appropriate.

One aspect in particular that needs to be considered while defining possible SOA service layers is their granularity. The following three factors are the most important in helping to determine the right granularity:

Business Suitability

Ideally, a service supports an atomic operation of a business task. It provides just enough functionality needed for that task. Whenever making it simpler makes it more general, this will increase the level of reusability of the service.

One service that handles a customer with addresses as well as all the orders the customer can place is likely too big. You probably cannot reuse it for some other business task that only involves handling a customer and no order lines (for example, for security reasons).

Performance and Size

Services are called remotely and therefore require a roundtrip, and the resulting performance overhead. You therefore want to minimize the calls to a service as much as possible, as long as that does result in services having too many responsibilities (see previous point). For example, rather than identifying a service that handles one order as well as another service that handles one order line, you identify one service that handles an order together with its order lines.

Furthermore, services are limited in the size of the messages they can process efficiently. When a message becomes too big, it will require too many resources to process. Transactions and State operations should, to the extent possible, be self-contained to reduce the risk that state get lost when a service fails to complete. When a service stores data this should be done in exactly one transaction and failure of a service should not result in corrupting the data. In other words, between two calls the data will always be in a valid state.

Integration Considerations

During this analysis, try and look at the number of partner links to maximize the business logic to have services that are self-contained. Try designing components that do not have a fine granularity.

There are several ways to approach the integration analysis for defining the integrations and the canonicals. One approach is to think of a hub and spoke model where there is a single canonical that can be used for the services. Another approach is to have a unique canonical for each service. A final approach may be to utilize an ESB (Enterprise Service Bus) architecture.

RA.023 Develop Use Case Model

When you have developed the Use Case Model during a project, the use cases should be verified against the Enterprise Repository for reuse. You should verify this as early as possible to see whether you can identify any duplicates or partly duplicates that you may reuse in your modeling efforts. It is often that the same or similar use cases are used in multiple projects. To prevent duplicates across the Enterprise, you should verify this as early as possible. You should perform this type of verification

multiple times during the process, as you might discover new aspects along the way, which may result in new mappings. You should classify your use cases following the use case model taxonomy. That makes it easier to identify any duplicates.

Review the Enterprise Requirements Management technique for further information about this.

During Service Identification & Discovery, the Use Case Model may be used as a basis for Functional Activity Analysis. Review the Service Identification and Discovery technique for more information about this.

PREREQUISITES

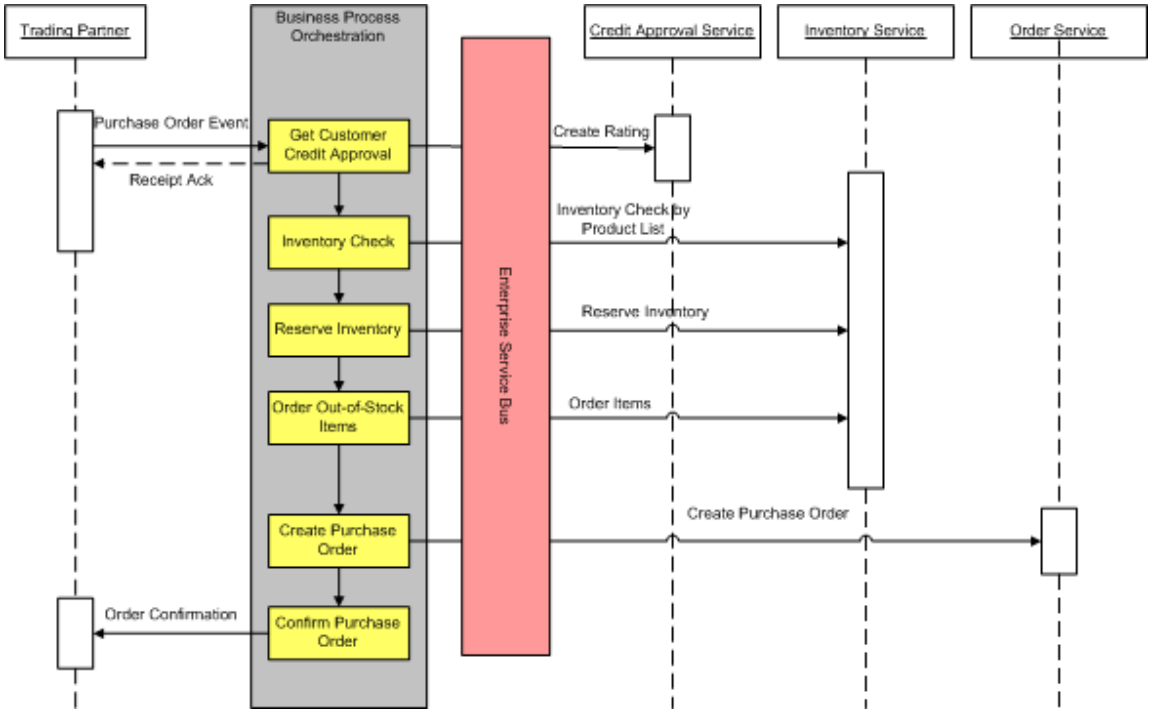
There are many prerequisites for this task and a simplified Use Case Model may be sufficient for Tactical and Pragmatic SOA Approaches. For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Business Use Case Model (RA.015)	Required	Optional	The Business Use Case Model is refined into the consistent Use Case Model.
System Context Diagram (RD.005)	Required	Optional	Use the System Context Diagram as a basis to identify new actors and use cases or to detail those identified in the Business Use Case Model.
Future Process Model (RD.011)	Optional	Optional	The Business Use Case Model and Future Process Model serve as input together with the initial Use Case Model to determine refined versions of use cases and additional use cases.
High-Level Business Descriptions (RD.020)	Required	Optional	The High-Level Business Descriptions are used as a basis to identify new actors and use cases or to detail those identified in the Business Use Case Model.
Current Process Model (RD.030)	Required	Optional	The Current Process Model is used as a basis to identify new actors and use cases or to detail those identified in the Business Use Case Model.
Current Business Baseline Metrics (RD.034)	Optional	Optional	The Current Business Baseline Metrics is used as a basis to identify new actors and use cases or to detail those identified in the Business Use Case Model.

RA.024 Develop Use Case Details

As Use Cases will provide the base for the design, it is strongly recommended to detail a use case with Business Process Diagrams using the BPMN (Business Process Modeling Notation), as this will be the base for the design of the business process to implement. A Business Process Diagram is similar to a UML Activity Diagram. BPMN has a clearly defined mapping to BPEL4WS.

At this point, enough details of the use cases should be available to begin identifying the services required to satisfy the use case. You therefore might consider utilizing business interaction diagrams (see Business Interaction Diagram below), which documents the interaction between the orchestration layer and services.



Business Interaction Diagram

These diagrams are very similar to UML Sequence Diagrams. In the center of the business interaction diagram is the business process or orchestration lane. To the left of the orchestration lane is the client of the orchestration service. For each interaction/event between the client and orchestration layer, the diagram indicates the services invoked and the operation called.

RA.025 Identify Candidate Services

This task is about identifying new service candidates and discovering reuse of existing services.

Consider the following guidelines order to identify the candidate services:

1. For those circumstances where the organization has not defined a SOA reference architecture, start with the SOA reference architecture defined by Oracle.
2. Use the guidelines and principles for each of the service layers that have been established by the reference architecture.
3. Leverage the Domain Model and business process models identified as part of the service requirements management tasks.
4. Record service candidates according to the organization's requirements. A spreadsheet is a good template to start with. Subsequent steps will want to add meta-data to each of the service candidates.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Future Process Model (RD.011)	Optional	Optional	Use this work product to identify candidate services.
Current Process Model (RD.030)	Required	Optional	Use this work product to identify candidate services.

Domain Model (RD.065)	Partial	Optional	Use this work product to identify candidate services.
Use Case Model (RA.023)	Required	Optional	Use this work product to identify candidate services.

As noted in the OUM task description for RA.025, you can identify candidate services from either domain models or business process models through the high-level use cases that emerge.

Another approach that is not explicitly mentioned in the OUM task description involves leveraging subject matter experts with deep domain knowledge to elicit candidate services. You can use the domain modeling or business process modeling exercise to elicit candidate services from such experts, or engage in ancillary techniques after the models are built such as CRC exercises originally used in OOAD. CRC exercises also work best when bounded by specific high level use cases, and thus fit well with the recommended service discovery flow in OUM.

RA.026 Populate Services Repository

Not mentioned in the task description for RA.026, is the strong link between the Design Time Service Repository and SOA Governance. The services repository can serve as an instrument of design time governance if it is also a metadata catalog. With appropriate tagging of business and technology assets such as design patterns and service descriptions, it is possible to use the repository with IDEs and run-time registries to implement governance policies. For clarity OUM separates the concepts of populating a design-time repository from registering services in a run-time registry. However, in practice the trend is to closely link the repository and registry.

Task Step 2 in RA.026 refers to defining a service template. Such a definition should include the following (these should be high-level definitions as each section will be defined in detail in the Design phase):

Item	Description
Service Name	The name of the service
Service Description	An overview description of the service
Service Type	Synchronous/Asynchronous
Functional Domain	Human Resources, Accounting, Manufacturing, etc.
Service Classification	As an example, you could use Component Layer, Data Layer, Business Layer and Workflow Layer. There are other ways of classifying are possible.
Service Process Rules	Detailed service processing logic
Message Formats	The inbound and outbound message formats. If applicable, define the fault message format for synchronous services.
Security Requirements	Any security requirements in relation to client authentication and authorization and data encryption
Applications	Identify the data sources (for example, application, database) this service must interact with to complete its functionality. Details of the application interface used for communication.
Service Dependency	Any other services this service depends on for functionality.
Human Workflow	If applicable, details of human workflow capabilities this service must provide.
Data Persistence	Setting up the dehydration tables
Performance Goals and Measures	Setting up the dehydration tables
Exception Handling	Describe any exceptions and exception handling logic. Define criteria for when faults are generated by the service.

RA.085 Validate Functional Prototype

In SOA implementation, a typical scenario will be BPEL with Workflow implementation. In this case, the human workflow work list application's user interface typically addresses human workflow within business process and reporting requirements. Complexities of the workflow interface need to be defined which will include custom screen development using BPEL workflow API. BPEL designers and developers should work with the ambassador users to verify and validate the functionality listed in the requirement for the functional prototype.

Back to Top

DESIGN (DS) PROCESS GUIDANCE

The Design process of a SOA implementation is focused on the development of services and the orchestration of those services. Ingrained in the SOA approach is the design and development of loosely coupled services that can be orchestrated and re-orchestrated, as business needs change. Since BPEL is standardized on WSDL, such adaptability is inherent within the SOA approach.

DS.040 Develop Design Architecture Description

The purpose of this task is to update the system's architecture to add design-related details that establish a set of components to be designed, to determine the structure of those components within the system at runtime, and to determine guidelines for proceeding with detailing the design of the identified components.

One step is to identify **Design Guidelines** that should be applied during Service Design. If there is a SOA Reference Architecture (EA.075) in place, then it should contain design principles to which each project should adhere. Use this as a starting point for your Design Guidelines. If a SOA Reference Architecture is not available, and there is limited availability fo other architecture artifacts, start with the OUM Reference Architecture guidelines available through the Envision Reference Architecture Planning view.

The next step is to determine the **Module View** for each service. Here, you determine all technical components that will be required for the service, such as XSD's, WSDL's, (reusable) Java functions you may need, and so on.

The SOA integration architect should present a holistic view of the architecture leveraging the software components provided by Oracle. Try to design each layer with the corresponding software component in mind.

For example, when you design the orchestration layer, you might want to consider ESB as the transport, transformation, and routing service component for that layer. Additionally, you might want to classify services depending on their logic and the business area they are serving. For example, you can classify services into infrastructure services, business logic services, utility services, EIS (Enterprise Information Systems) Services which typically includes CRM and ERP systems, process control services which typically represents automated and manual operations, security services, etc.

This document should provide high-level architectural details for the logical (module) view and the physical (execution) view. The logical view addresses the primary modules within the SOA solution including app servers, BPEL engines, LDAP servers, databases, web services manager, enterprise monitor components, and services. The physical view maps the various components defined in the logical view to the hardware upon which they will be deployed. Other items addressed in this document are in areas of high availability (app server clustering) and scalability (clustering and load balancers).

Note that the JMS servers should be defined. Consider BPEL server clustering, and networking firewalls for both the external and internal network, using a proxy server. If AQ is to be used then AQ affinity should be considered for high performance and the initial size of the AQ should be defined. Consider using RAC. Many customers such as financial institutions will prefer to have different layers of firewalls (in some cases, at least 4). Such security and performance considerations need to be accommodated. Also, some customers may require keeping certain data in certain geographic locations. Such factors need to be considered here, also.

DS.070 Define SOA Implementation Strategy

In this task, you define how and when your projects services are going to be released. You determine if they are going to be released as one or several groups or individually and which service(s) should be released first.

This task is highly influenced by the requirements that are defined in the enterprise SOA Reference Architecture (EA.075) and Project Reference Architecture (RA.019). However, if this is being done in a tactical manner (in absence of EA.075 or RA.019), look at the requirements pertaining to the initial/project-level services that are implemented at the first stage. Work with the customer project manager to prioritize these needs and create a plan based on the tasks identified. When creating the implementation plan, consider the applications/projects that are dependent on these services and their schedules.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Technology Reference Architectures (ENV.EA.075)	Partial	Optional	If available, use the Technology (SOA) Reference Architecture to gather details on the enterprise version of the SOA Reference Architecture and Roadmap for guidance on developing the SOA Reference Architecture and Roadmap at the project level.
Project Reference Architecture (RA.019)	Partial	Optional	Use the Project (SOA) Reference Architecture to understand the service deployment strategy, if any.
Architecture Description (RD.130/RA.035/AN.040/DS.040)	Partial	Optional	The design portion of this work product (DS.040) produces the service contracts upon which the service design should be based.

Back to Top

IMPLEMENTATION (IM) PROCESS GUIDANCE

IM.010 Develop Functional Prototype

Development of a functional prototype normally is done iteratively. Primary focus is on the development of services and business process orchestration flow. Since services are self-contained, 'black box' solutions, service development can be distributed across development staff for concurrent development.

To support an iterative design of the business process, service skeletons can be developed early on. A service skeleton is an actual service that implements the service interface but returns hard coded or configurable reply data. A common approach is to build skeleton services that read XML reply data from a file. This allows the business process developers to build a full orchestration solution that communicates with the 'skeleton' services. The orchestration developers can make changes to the reply data as needed for developing and unit testing the flow logic. Once actual service implementations are ready, they can be incrementally incorporated into the business process.

During the implementation, consider creating common testing components that can used for handling inbound and outbound messages from the external system to the integration. Some items to consider:

- Develop a way to test a message event that is executed from an external system and a way to trigger an external system.
- Develop simple components for networking tests to external systems.
- Define and set up the integration option data, which is used to change the message at runtime.
- Know and use pre-developed integration patterns.
- Put together a library of commonly used components.
- Set up simple testing prototypes for use with all adapters.

IM.050 Implement Components

In this task, you implement the components of your application. A component is the physical packaging of model elements, such as design classes created in the Design process. Components have a trace relationship to the model elements they implement. Some standard types of components include executables, files containing source

code, library etc. This task focuses on developing and maintaining components (fixing defects when the class has been tested).

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Reviewed Design Model (DS.160)	Required	Required	The Reviewed Design Model provides the necessary input to develop the components for the application.

IM.060 Perform Component Review

Unit testing can occur at several levels. In addition to testing an individual component, the following levels should be considered:

- **Service Unit Testing** - Services can be unit-tested by invoking the services through the service interface definition. A testing framework that provides for programmatic and repeatable testing should be pursued. The name is a little bit misleading here, but an automated testing framework like Apache JUnit plus Cactus can be used for this type of testing. In principle Service Unit Testing is the peer of the business system-testing task that governs Component Integration Testing.
- **Orchestration Testing** - Utilizing actual or 'stub' services, the services orchestration flow can be tested for correct logic design and error handling. It is very important that the test suite adequately cover negative test scenarios for the end-to-end integration. A messaging layer is common in integration implementations and full negative testing should be applied to minimize the chance of messages getting “backed up” on a message queue and preventing further processing. Using an automated testing framework like Apache JUnit with Cactus can also support Orchestration Testing. In principle, Orchestration Testing is the peer to System Testing (when restricted to services of the system being developed) and Systems Integration Testing (where it involves services of external systems).

Back to Top

TESTING (TE) PROCESS GUIDANCE

TE.010 Develop Testing Strategy

In a Strategic SOA approach, identify requirements related to test environment(s), how tests are performed, and acceptance criteria in a centralized or distributed deployment environment.

In a Pragmatic SOA approach, depending on the direction set in the earlier phase (Service Analysis phases), find the requirement, if any, for a centralized test environment. If available, identify the testing strategy, collect metrics, and define acceptance criteria.

In a Tactical SOA Approach, the requirements needs to address how the tests are performed, what metrics are collected, and the acceptance criteria in a project deployment environment.

The purpose of this task is to develop the project direction as it relates to testing. This should include how testing should be performed in the various stages of the project, which tools should be used (if any) and how, what metrics should be collected, how test data should be produced, and what testing environments should be used in the tests. The strategy should also define the acceptance criteria of the solution.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Configuration Management Plan and Processes (PJM.CM.010)	Required	Required	The environments that will be used for testing should be synchronized with the promotion levels defined in the Configuration Management Plan and Processes.
Business and System Objectives (RD.001)	Required	Required	The Business and System Objectives states the objectives to be met by the new system.
Use Case Specification (RA.024)	Required	Tentative	The Use Case Specification provides use case scenarios that lead to test scenarios that need to be tested in the various tests, and therefore is an input to the Testing Strategy.
Testing Requirements (TE.005)	Required	Required	The Testing Strategy must be in line with the Testing Requirements as the strategy describes how the requirements will be satisfied.
Data Acquisition and Conversion Requirements (CV.010)	Required	Required	The Data Acquisition and Conversion Requirements indicate which business objects will be converted and where data may be used during testing.

A testing strategy for the integration components within SOA architecture should be defined. As with any testing strategy, the goal is a repeatable process that provides full test coverage of the components, services, and processes that compose the SOA framework. Adding to the testing types that have been identified in the core method, a strategy should also cover the following:

- Service Testing - Services can be tested by invoking the services through the service interface definition. A testing framework that provides for programmatic and repeatable testing should be pursued. When Service Unit Testing has been addressed during the task IM.060 Perform Quality Assurance on J2EE Components testing at this stage might restrict itself to reviewing the unit tests of IM.060 and enhance them wherever necessary.
- Business Process Testing - Business process testing typically falls in the system integration test phase. As with the unit testing identified above, it is highly important that the test suite adequately covers negative test scenarios for the end-to-end integration. A messaging layer is common in integration implementations, and full negative testing should be applied to minimize the chance of messages getting “backed up” on a message queue and prevent further processing. When Orchestration Testing has been addressed during the task Perform Component Review (IM.060) on the J2EE Components using an automated testing framework, this can be used as a starting point for Business Process Testing. Depending on the coverage provided by that test, Business Process Testing might be restricted to reviewing the unit tests and enhancing them wherever necessary.

TE.015 Develop Integration Test Plan

In this task, you develop the Integration Test Plan, which you use as a guide to perform the integration tests that are performed during the iterations in the Elaboration and Construction phases. The Integration Test Plan is produced iteratively along with the development iterations to ensure that the plan reflects what is actually developed in the iterations

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Baseline Project Workplan (PJM.WM.010)	Optional	Optional	Use the Baseline Project Work plan is understand how integration testing activities have been planned
Quality Management Plan (PJM.QM.010)	Required	Optional	Use the Quality Management Plan to see how quality will be ensured and how that is related to testing.
Testing Requirements (TE.005)	Required	Required	Use the Testing Requirements and Testing Strategy to understand how the integration tests should be performed.
Testing Strategy (TE.010)	Required	Tentative	Use the Testing Requirements and Testing Strategy to understand how integration tests should be performed.

Domain Model (RD.065)	Required	Optional	Use the Domain Model to identify and define test types, test items, and features to be tested.
Use Case Model (RA.023)	Required	Optional	Use the Use Case Model to identify and define test types, test items, and features to be tested.
Conversion Component Designs (Initial Load) (CV.040)	Required	Optional	Use the Conversion Component Designs (Initial Load) to identify and define test types, test items, and features to be tested.
System Management Guide (TA.100)	Required	Optional	Use the System Management Guide to identify and define test types, test items, and features to be tested.

TE.020 Develop Unit Test Scripts

In this task, you develop the scripts to test individual components. The tests validate that the components inputs, outputs, and processing logic function as required. The task is mainly performed in parallel with the actual development of the components. Thinking through how the component should be tested increases the understanding of what the component should do or produce. Also, the component can be immediately tested when developed. As part of the task, you also prepare common checklists that are used in performing unit testing for each type of primary or supporting component. This is done prior to developing individual unit test scripts for each unit test.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Reviewed Design Model (DS.160)	Required	Required	The Reviewed Design Model contains the use case realization - design and the designed classes. The Business Rules Design describes the functionality and business rules that should be implemented. They are used as your primary guide in developing unit test scripts.
Reviewed Requirements Specification (RD.150)	Required	Required	The Reviewed Requirements Specification captures the original requirements and helps you understand how the component being tested fits into the larger business process.
Testing Requirements (TE.005)	Required	Required	The Testing Requirements provides the requirements for testing.
Testing Strategy (TE.010)	Required	Optional	The Testing Strategy provides the testing approach and strategy.

TE.030 Perform Unit Test

In this task, you test application components on an individual basis to verify that the inputs, outputs, and processing logic of each application component functions without errors. Unit testing is most often performed in the development environment, but it is also possible to use a separate testing environment. The task is performed in parallel with the actual development of the components.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Testing Requirements (TE.005)	Required	Required	The Testing Requirements provides the requirements for testing.

Testing Strategy (TE.010)	Required	Optional	The Testing Strategy provides the testing approach and strategy.
Unit Test Scripts (TE.020)	Required	Required	The Unit Test Scripts provides the prerequisite seed data, the detailed steps, and the expected results of the test.

TE.040 Perform Integration Test

In this task, you perform the appropriate integration tests during each development iteration, both in the Elaboration and Construction phases. Test the components based on the plan outlined in the Integration Test Plan using the scenarios described in the Integration Test Scenarios.

PREREQUISITES

For SOA implementations, consider the following for this task:

Prerequisite	Pragmatic	Tactical	Usage
Integration Test Plan (TE.015)	Required	Optional	This work product provides the plan of how to perform integration tests.
Integration Test Scenarios (TE.035)	Required	Optional	This work product provides the scenarios that should be tested.
Integration Test Environment (TE.038)	Required	Optional	This work product contains all components that should be tested as part of integration testing.

Back to Top

TECHNICAL ARCHITECTURE (TA) PROCESS GUIDANCE

In case of a SOA, the Technical Architecture process requires a sharp focus on the infrastructure for a SOA deployment project. By exposing its business systems as services, a business can expose itself to unpredictable load levels and a hostile security environment. The importance of the technical infrastructure can therefore hardly be overstated.

The architecture approach of an SOA integration implementation should follow the guidance as provided in the SOA introduction.

TA.020 Define Technical Architecture Requirements and Strategy

Adding to what has been described in the task guidelines of the core method, during this task you will focus on the following subjects:

- Functional/pictorial overview
- Technical operations overview
- Fault handling and retry logic (foreign system unavailability)
- A detailed technical description (data mappings, APIs used, database interactions)

For integration solutions requiring B2B, the trading partner agreement must be captured for each trading partner with which the solution will interact. The document should address such B2B topics as the B2B protocol standard used, message formats, message types, partner interactions, contact information, and SLAs.

Key performance indicators should be defined that will drive management BAM (Business Activity Monitoring) dashboards. Definition of the KPI parameters and resulting graphs/reports will highlight any extra data they needs to be captured as part of the normal business events.

The business events identified in the Requirements Analysis section should be utilized for capacity planning. Useful input for hardware sizing estimates is a table of

business events that must be processed by an integration solution. An inventory table of all events should include the following columns at a minimum:

Column	Description
Message Event Name	A user assigned name for the message event.
Message Type	A description of the message type/format (that is, EDI X12 855, CIDX Order Create).
Average Size	Average size of the message event.
Transformation Complexity	Transformation complexity indicator: • Low - less than 100 node transformation • Medium - 100 to 500 node transformation • High - 500+ node transformation This assignment is subjective and data transformation complexity should be considered.
Source System	The source system where the business event originates.
Source Protocol	The transfer protocol (that is, FTP, HTTP, SOAP, AS2) for the source event.
Source Format	A description of the native format of the source event.
Destination System	The destination system where the event must be delivered. Note: Multiple destination systems may need to be defined.
Destination Protocol	The protocol to communicate with the destination system.
Destination Format	The protocol to communicate with the destination system.

Also, for integration solutions that require interfacing with external trading partners, the following information is useful for capturing the volumes required by a B2B solution. This information is typically captured in a spreadsheet.

Column	Description
Partner Name	Name of partner.
Contact Info	Contact information for trading partner. Should provide management, development, and support staff.
Exchange Protocol	RNIF/AS2/Generic
Document Protocol	RosettaNet, EDI X12, EDI EDIFACT, Custom
Transport Protocol	HTTP/HTTPS, Email, File/FTP
Message Type	Message types to be handled as defined in the document protocol.
Message Direction	Direction of message type (referenced from customer company).
Message Volume	Volume of messages; peak times; also should address growth scenarios over 2 to 5 years.
Message Size	Average message size; largest message size.
Non-Repudiation	Non-repudiation requirements.

Service Level Agreements	Time to acknowledge requirements, retry limits.
Security Requirements	Define security requirements for message exchange. Address MIME and S / MIME.

TA.030 Define Integration Requirements and Strategy

By following the methodology and tasks listed in this SOA Integration Supplemental Guide, practitioners will have a solid ground upon which they can perform requirements analysis and create an integration strategy. For example, practitioners should be able to perform service identification by the ways of top-down domain decomposition and bottom-up event/process tracing. The deliverable for those tasks will be the consolidated version of an Enterprise service portfolio and candidate service table. By performing detailed use case development according to the service table, practitioners will be able to capture the relevant services and deliver use case realizations accordingly. Once you perform the RD tasks listed in this guide, you will then be able to define a clear high-level, definition of the SOA enterprise integration strategy. Even if your project does not require an Enterprise service portfolio, but is a more typical Integration effort, this OUM guidelines should be used by the practitioner to ensure all requirements and strategy are well articulated ahead of actual implementations.

TA.090 Develop Security and Control Strategy

Security requirements for integration need to address the following areas:

- Services - Define the client authentication and authorization requirements for service access. Services can exist as web services, EJBs, JMS, etc. and appropriate security access requirements need to be defined.
- Business Process/Orchestration - Similar to services defined above, business processes can exist as services to other clients and applications for enforcement of central business rules. Appropriately exposing these processes to authorized users should be addressed.
- B2B - Define the partner authentication and authorization requirements.
- Data Encryption - The requirement to encrypt data applies to all of the previous sections. Certain information such as passwords or Social Security Numbers may need to be encrypted for privacy. Or perhaps the entire message may need to be encrypted. Even if non-sensitive data is being handled, guaranteeing data integrity with the use of digital signatures may also be required. This is typical in B2B integration scenarios.

Back to Top

DATA ACQUISITION AND CONVERSION (CV) PROCESS GUIDANCE

For most integration projects, Data Acquisition and Conversion tasks are core tasks rather than optional. However, in integration projects, this work is often done for each disparate integration point (whether A2A or B2B) so that there may be dozens of data conversions (often called data mappings in integration projects). In addition, these conversions or mappings are executed in-line every time an integration point is used.

Data Acquisition and Conversion for integration projects is a much more extensive aspect of projects than in typical development projects which likely required only a one time batch conversion. In fact, using SOA and relevant support technologies to manage continuous and complex integration data mapping is one of the major justifications for SOA Integration. Each of the following tasks will need to be completed for each integration point.

- **CV.020 Define Data Acquisition, Conversion and Data Quality Strategy**
  CV.030 Define Conversion Environment (Initial Load)
  CV.040 Design Conversion Components (Initial Load)
  CV.055 Implement Conversion Components (Initial Load)

CV.060 Perform Conversion Component Unit Test (Initial Load)

You should follow the OUM guidance for these tasks.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

SERVICE-ORIENTED ARCHITECTURE (SOA) PROJECT DELIVERY SERVICE INTEROPERABILITY GUIDELINES

Back to View

In order to provide or consume a "Business Service", the involved systems/partners must agree on the interaction pattern with the corresponding business semantics. Agreement must also exist for the message binding contracts and policy profiles for each service. This document provides guidelines, patterns and templates to capture all the necessary information related to service interoperability.

SERVICE SCOPE

This section introduces common high-level scenarios that illustrate Web Service solutions based on the scope of the service from the enterprise boundary perspective. This scope based Web Service scenario classification would be helpful to derive requirements as well as the possible patterns that can be utilized as a solution corresponding to a particular scenario.

Public Service

In this scenario, Web Services are exposed to the entire world and depending on the application, the security requirements will vary greatly ranging from no explicit security to high protection and prevention methods.

Intra-Enterprise Service

In this scenario, Web Services are exposed within an enterprise for other applications to consume.

Inter-Enterprise Service

In this scenario, Web Services are exposed to partners to conduct business-to-business transactions.

Federated Service

In this scenario, a set of Web Services is exposed by partners to collaborate and conduct a business transaction in a federated manner.

Back to Top

INTEROPERABILITY

Services are said to be interoperable only when the involved systems and partners understand both Business and Technology semantics.

Business Layer Policies

At the business layer, the domain object and the action are the 2 most important policies. Domain objects are nouns such as ‘Customer’, ‘Purchase Order’ etc. Actions are verbs such as ‘Create’, ‘Approve’ etc.

Business Message

The business messages are ‘domain objects’ (nouns) and are modeled using the following methods of achieving interoperability:

- **Distributed Point-to-Point** - In this approach, each service (corresponding to a system) has its own proprietary semantic meaning and semantic translation is performed in a point-to-point manner local to a service. In other words, every service will be interoperating with every other service using its own internal transformation in a point-to-point manner.
- **Centralized Point-to-Point** - In this approach, each service (corresponding to a system) has its own proprietary semantic meaning and semantic translation is performed in a point-to-point manner but centrally. In other words, every service will be interoperating with every other service using the central located transformation logic in a point-to-point manner.
- **Distributed Hub and Spoke (Canonical Data Model)** - In this approach, each service (corresponding to a system) has its own proprietary semantic meaning but is mapped to a logical data model, which can be initiated as a physical federated model or a canonical data model. In other words, the Hub is virtual and is modeled as a canonical message and spokes will do the semantic transformation internally.
- **Centralized Hub and Spoke (Master Data Management)** - In this approach, each service (corresponding to a system) has its own proprietary semantic meaning but is mapped to a single data model managed by a specialized application such as Master Data Management (MDM systems. In other words, the Hub is physical and is responsible for maintaining single version of truth.

Business Message Versioning

The versioning of business messages can be done using the following methods:

- Type based: The versioning scheme is part of the domain object name. If the message is modeled using XSD, the target namespace for the domain object type could signify the version.
- Instance based: The versioning information is sent as part of the message. If the message is modeled using XSD, then the corresponding XML message will carry the version information as part of an element or attribute.

A hybrid versioning by utilizing both type based (for major version number) and instance based (for minor version number) may be used as well.

Versioning could use the date format such as YYYY/MM/DD.

Business Transaction

The business transactions are ‘actions’ (verbs) that are modeled using the timing of interactions between the involved partners/systems/services.

Message Exchange Patterns

The best practice is to have one message exchange pattern, such as the following, per service and business processes should be used for orchestrating more than one service to model complex message exchange patterns.

Following are some of the common message exchange patterns for a coarse grained service.

- Synchronous-Query-Response-Fault - This pattern should be used for scenarios where a response be returned for a query and the state of the system must not be changed.
- Synchronous-Request-Response-Fault - This pattern should be used for scenarios where a response be returned for a request and the state of the system may be changed.
- Synchronous-Request-Ack-Fault - This pattern should be used for scenarios where an acknowledgement be returned for a request to indicate that the provider received the request.
- Asynchronous-One-Way - This pattern should be used for notification or quote like scenarios where the delivery reliability is not mandatory.
- Asynchronous-Request-Response - This pattern should be used if the processing time of returning a response for the corresponding request is unpredictable.
- Asynchronous-Request-Ack - This pattern should be used if the processing time of returning an acknowledgement for the corresponding request is unpredictable.
- Asynchronous-Request-Ack-With-Timeout - This pattern should be used if the processing time of returning an acknowledgement for the corresponding request is unpredictable and if the consumer insists on getting the acknowledgement within a specified period of time.
- Asynchronous-Request-Ack-With-Timeout-Notification - This pattern should be used if the processing time of returning an acknowledgement for the corresponding request is unpredictable and if the consumer would like to be notified when the acknowledgement is not received within a specified period of time.
- Asynchronous-Request-With-Optional-Ack-With-Timeout - This pattern should be used if the processing time of returning an acknowledgement for the corresponding request is unpredictable and if the consumer insists on getting the acknowledgement optionally within a specified period of time.
- Asynchronous-Request-Response-With-Optional-Ack-With-Timeout - This pattern should be used if the processing time of returning a response for the corresponding request is unpredictable and if the consumer insists on getting the response optionally within a specified period of time.

ACID Transaction

Some business services required to perform atomic transactions with ACID properties and are usually synchronous in nature.

The service will lock the appropriate resources while conducting the ACID transactions.

Above mentioned synchronous message exchange patterns should be used by the service if it requires conducting ACID transactions.

Long Running Transaction

Some services might require running for hours, days, months or years before concluding a business transaction.

The service will release the locks and hence isolation property of ACID is relaxed.

Above mentioned synchronous message exchange patterns should be used by the service if it requires conducting long running transactions.

Technology Layer Policies

At the technology layer, standards based policies related to transport, security etc. are important for higher interoperability.

Following are some of the common technology related components that are related to service interoperability.

Transport

The most common transport mechanisms used by service endpoints are HTTP or messaging middleware such as JMS, MQSeries etc.

In order to be interoperable, messaging/transport protocol version and headers need to be defined and agreed upon, such as ‘SOAPAction’ HTTP 1.0 header in case of SOAP based service invocation.

HTTP transport is not reliable, whereas messaging middleware such as JMS provides reliable delivery of messages. But HTTP is more prevalent and firewall friendly.

The best practice is to use HTTP for interacting with external partners and messaging middleware for internal partners/systems.

Batching

Some business processes such as monthly billing require batching capability instead of real-time interaction. In some other cases, business systems are inherently batch-oriented such as mainframe applications. In some cases, batching is used by small IT organizations to avoid higher IT and communication costs.

Services may have to support batch-oriented interaction in addition to real-time interaction. In order to support batching, the messaging model must support the following elements either implicitly or explicitly:

- Batch-id
- Sequence-number and
- Batch-termination-flag

Reliability

The Technology Layer requires that the messaging systems of business partners that are involved in consuming and providing business services must support reliability in terms of Guaranteed Delivery, Guaranteed Processing and Visibility.

Guaranteed Message Delivery

Guaranteed Message Delivery is a mechanism to ensure that the message reaches the incoming message port of a service based on the QOS parameters such as only once or at-least once.

If HTTP is used as a transport, then higher-level protocols such as WS-ReliableMessaging must be used, if guaranteed message delivery is a requirement.

Guaranteed Message Processing

Guaranteed Message Processing is a mechanism to ensure that the message not only reaches the incoming message port but that it is correctly processed by the destination business service. It provides the inherent ability to ensure that a business service dialogue is completed to the confirmed satisfaction of all parties involved in that business transaction.

Guaranteed Business Processing

Guaranteed Business Processing can be achieved by sending a ‘Business Response or Business Acknowledgement’ message/MEP for every ‘Business Request’ message/MEP. Integration infrastructure does not need to implement business-level guaranteed processing functionality within itself, but will be implemented specific to business scenarios at the enterprise application level.

Visibility

Visibility gives a global view about the consistency and the current state of the various systems that were involved in business service execution.

Business users need visibility at the business document level and technical production support users need visibility at both business document level as well as at the messaging level including detailed logs and audits for debugging and managing. Any other visibility functionality other than related to guaranteed-delivery and guaranteed-processing need not be part of service infrastructure or integration application.

For example, capturing the number of successfully processed orders from a specific partner or system is a valid visibility requirement from a service point of view. Also, resending the messages that are not delivered due to, say, unavailability of the target system is part of the visibility/monitoring and management tool.

The following are some of the meta-data elements that would be captured and managed by the visibility tool:

- Message ID – a globally unique ID
- Related Message ID – correlation ID
- From Partner Profile ID – points to the action specific partner profile

- To Partner Profile ID - points to the action specific partner profile
- Application Environment – environment context such as Production, Test etc.
- Timestamp – created and received date time
- Message Exchange Pattern – MEP used by the service
- Business Document Context (Business Object IDs) – a composite key identifying the business document uniquely
- Message Blob – sent or received message

The above attributes address only the technology level visibility and would be useful only to the system administrators. If business process level visibility in terms of KPI is required, then BAM component must be used.

Security

Hackers and cyber terrorists are launching security attacks with increasing frequency and sophistication. The traditional approach to security - a firewall combined with an anti-virus - is incapable of protecting from today's advanced threats.

However, a formidable defense can be erected by implementing security using a layered approach as mentioned below. By selectively installing security measures on five levels (as listed below) within the IT environment, digital assets can be adequately protected and the risk of exposure to a catastrophic network breach can be greatly reduced.

The layered-security approach centers on maintaining appropriate security measures and procedures at five different levels within an IT environment:

1. Perimeter
2. Network
3. Host
4. Application
5. Data

In the context of SOA based integration, the technology layer requires and recommends that the messaging systems of business partners that are involved in consuming and providing business critical services are fully secured by preventing unauthorized users or systems from deliberately or accidentally accessing the business services.

In the context of SOA based integration, security could be applied at two different layers as follows:

- Point-to-Point Transport Level Security
- End-to-End Business Message Level Security

In most intra-enterprise business scenarios, transport level security is sufficient. In some inter-enterprise business scenarios, especially when an intermediary node is involved, business message level security is desired.

Following are the security features that could be implemented or deployed for securing business services, depending on specific business scenarios and requirements:

- Authentication
- Authorization
- Confidentiality
- Integrity
- Non-Repudiation

From the 'standards' and 'best practices' perspective, for inter-enterprise services, SSL is preferred for transport level security and WS-Security is preferred for message level security. SSL offers the security features of authentication, confidentiality and integrity at the transport level, whereas WS-Security offers all of the above-mentioned security features at the message level.

Authentication Patterns

Authentication is the process of identifying an individual or system using the credentials of that individual or system.

Following are the 2 high level patterns of authentication:

1. Direct Authentication
2. Brokered Authentication

Both the Direct Authentication pattern and the Brokered Authentication pattern focus on the relationships that exist between a client and service participating in a Web service interaction.

When both the client and service participate in a trust relationship, that allows them to exchange and validate credentials including passwords, direct authentication can be performed. Direct authentication requires the presentation of credentials, which are typically a user name and password. The service uses these credentials to authenticate the request directly.\

In a situation where the client and service do not share a direct trust relationship, broker authentication can be performed. Credentials are used to authenticate with the broker, which issues a security token. The security token is then used to authenticate with services.

Depending on the type of token, security protocol standard etc, following are the 3 kinds of brokered authentication mechanisms that are widely used:

1. X.509 PKI
2. Kerberos
3. Security Token Service (ex, SAML)

Authorization Patterns

Following are the 2 different methods of performing authorization:

- Role-based
- Resource-based

Role-based authorization is used to associate clients and groups with the permissions that they need to perform particular functions or access resources. When a user or group is added to a role, the user or group automatically inherits the various security permissions. Role-based authorization can be declarative or imperative.

Resource-based authorization is performed declaratively on a resource, depending on the type of the resource and the mechanism used to perform authorization. Resource based authorization can be based on:

- Access Control Lists (ACLs) or
- URLs

Individual resources are secured using ACLs that specify the type of operation a particular security principal or group to which a security principal belongs can perform on the object.

URLs are declared with permissions for the URL to define who is authorized to access the URL in question.

Message Protection Patterns

Web services send and receive plaintext messages over standard Internet protocols such as HTTP. Such plaintext messages can be intercepted by an attacker and potentially viewed or even modified for malicious purposes. By using message protection, you can protect sensitive data against threats such as eavesdropping and data tampering.

Message protection can be divided into three main categories:

- Data Integrity

Data Origin Authentication
- Data Confidentiality

Data integrity is the verification that a message has not changed in transit.

Data Origin Authentication takes data integrity a step further and supports the ability to identify and validate the origin of a message.

Data Confidentiality is the encrypting of message data so that unauthorized entities cannot view the contents of the message. Data confidentiality can be applied in two steps:

1. Encrypting the data. In this step, the sender converts plaintext to cipher text, rendering it unintelligible to parties other than the intended recipient.
2. Decrypting the data. In this step, cipher text is rendered intelligible to the intended recipient by converting it back to plaintext.

Two types of cryptography can be used to provide data confidentiality: symmetric or asymmetric.

With symmetric cryptography, both the sender and recipient share a key that is used to perform both encryption and decryption. Symmetric cryptography is commonly used to perform encryption. It also provides data integrity when symmetric keys are used in conjunction with other algorithms to create Message Authentication Codes (MACs). Some of the more common symmetric cryptography algorithms include Rijndael (AES) and Triple DES (3DES). These algorithms are designed to perform efficiently on common hardware architectures.

Symmetric cryptography is comparatively simple in nature, because the secret key that is used for both encryption and decryption is shared between the sender and the recipient. However, before communication can occur, the sender and the recipient must exchange a shared secret key. In some cases (such as SSL), asymmetric cryptography can be used to ensure that the initial key exchange occurs over a secure channel.

With asymmetric cryptography (also known as public key cryptography), the sender encrypts data with one key, and the recipient uses a different key to decrypt ciphertext. The encryption key and its matching decryption key are often referred to as a public/private key pair. The public key of the recipient is used to encrypt data. It can be openly distributed to those who want to encrypt a message to the recipient. The private key of the recipient is used to decrypt messages, and only the recipient must be able to access it. The most commonly used asymmetric algorithm is the RSA algorithm.

In the context of SOA for integration, it is important to agree on the cryptography algorithms to be used between service providers and consumers.

Data Origin Authentication enables the recipient to verify that messages have not been tampered with in transit (data integrity) and that they originate from the expected sender (authenticity).

In cases where the client denies having performed the action (non-repudiation), you can use digital signatures to provide evidence that a client has performed a particular action that is related to data. Digital signatures can be used for non-repudiation purposes, but they may not be sufficient to provide legal proof of non-repudiation. By itself, a digital signature is just a mechanism to capture a client’s association to data. In cases where data has been digitally signed, the degree to which an individual or organization can be held accountable is established in an agreement between the party that requires digital signatures and the owner of the digital signature.

Two types of signatures can be used to sign a message: symmetric and asymmetric.

A symmetric signature is created by using a shared secret to sign and verify the message. A symmetric signature is commonly known as a Message Authentication Code (MAC). A MAC is created by computing a checksum with the message content and the shared secret. A MAC can be verified only by a party that has both the shared secret and the original message content that was used to create the MAC.

The most common type of MAC is a Hashed Message Authentication Code (HMAC). The HMAC protocol uses a shared secret and a hashing algorithm (such as MD5, SHA-1, or SHA-256) to create the signature, which is added to the message. The message recipient uses the shared secret and the message content to verify the signature by recreating the HMAC and comparing it to the HMAC that was sent in the message.

An asymmetric signature is processed with two different keys; one key is used to create the signature and the other key is used to verify the signature. The two keys are related to one another and are commonly referred to as a public/private key pair. The public key is generally available and can be distributed with the message; the private key is kept secret by the owner and is never sent in a message. A signature that is created and verified with an asymmetric public/private key pair is referred to as a digital signature.

The algorithm that is most commonly used to create a digital signature is the Digital Signature Algorithm (DSA). DSA uses the public/private key pairs created for use with the RSA algorithm to create and verify signatures.

For both signing and encryption purposes, asymmetric keys are often managed through a Public Key Infrastructure (PKI). Information that describes the client is bound to its public key through endorsement from a trusted party to form a certificate. Certificates allow a message recipient to verify the private key in a client's signature using the public key in the client's certificate.

Resource Access Patterns

Web services represent a programmable interface that applications often use to access resources, such as the local file system, databases, or other Web services. Distributed applications can consist of multiple interacting Web services, so you have to consider the needs and constraints of the entire application instead of focusing on a single point of interaction. Authentication, authorization, and auditing, along with other environmental and operational requirements (such as scalability requirements at each resource access interface), should combine to influence the security solution.

The following methods can be used to access resources:

- Impersonation
- Delegation
- Protocol Transition
- Trusted Subsystem

Impersonation is the act of assuming a different identity on a temporary basis so that a different security context or set of credentials can be used to access a resource. When accessing local resources, such as a file in the local file system, you need only the security context. However, when accessing resources that require authentication, such as a Web service or database, credentials are required.

Delegation is not the same as impersonation; however, it requires impersonation to work. This is a process where the service account is allowed to access a remote resource on behalf of another account, which is typically the client accessing a service. Impersonation is required so that the service account can access the credentials of the account being delegated and send those credentials to the remote resource. For delegation to work, the account being delegated must also be configured to allow delegation.

Protocol transition is a process where the service account transitions an identity that was authenticated using a non-native protocol into a native security context. This can also be used to implement a trusted subsystem by using the service account's identity instead of the client's identity to access resources.

Trusted Subsystem is a process where a trusted business identity is used to access a resource on behalf of the client. The identity could belong to the service account or it could be the identity of an application account created specifically for access to remote resources. There are many different reasons for using a trusted subsystem, but the most common reason is to take advantage of resource sharing techniques, such as connection pooling associated with database connections.

Service Boundary Protection Patterns

'Message Protection Patterns', described how to provide protection against data tampering and unauthorized access to message content. However, in many cases you will need to provide additional protection at the service's boundary to:

- Protect Web services against malformed or malicious content.
- Ensure that when a Web service operation fails you do not accidentally reveal confidential information in the Fault message that is returned.
- Prevent an attacker from intercepting a message and replaying it to force a Web service operation to execute multiple times.

Service Boundary Protection includes the following:

- Message Replay Detection
- Message Validator
- Exception Shielding

Message Replay Detection: When the Web service processes the messages, data is updated or business processes are initiated. If a message that is intended for one of these Web services is intercepted and replayed, there is a risk that the same operation might be performed multiple times. Solution is to cache an identifier for incoming messages, and use message replay detection to identify and reject messages that match an entry in the replay detection cache. Message replay detection requires that individual messages can be uniquely identified.

Message Validator: A Web service interacts with other applications over a network. Incoming data may be malformed and may have been transmitted for malicious purposes. There is also a risk of injection attacks, where data from incoming messages is tampered with to include additional syntax. Assume that all input data is malicious until proven otherwise, and use message validation to protect against input attacks, such as SQL injection, buffer overflows, and other types of attacks. The message validation logic enforces a well-defined policy that specifies which parts of a request message are required for the Web service to successfully process it. It validates the XML message payloads against an XML schema (XSD) to ensure that they are well-formed and consistent with what the Web service expects to process. The validation logic also measures the messages against certain criteria by examining the message size, the message content, and the character sets that are used. Any message that does not meet the criteria is rejected.

Exception Shielding: Use the Exception Shielding pattern to sanitize unsafe exceptions by replacing them with exceptions that are safe by design. Return only those exceptions to the client that have been sanitized or exceptions that are safe by design. Exceptions that are safe by design do not contain sensitive information in the exception message, and they do not contain a detailed stack trace, either of which might reveal sensitive information about the Web service's inner workings.

Oracle AS10g and BPEL PM provides most of the common security infrastructure and OWSM provides a policy driven security features for advanced deployments. The above patterns must be considered for a given service before deciding on using the Oracle SOA Suite components.

Standards

Industry standards related to the above design patterns must be considered for higher service interoperability.

Specific standards supported by AS10g, BPEL PM and the Oracle Web services stack are listed below: (MOTM and WS-ReliableMessaging standards will be supported in AS11g)

HTTP 1.0 - http://www.w3.org/Protocols/HTTP/1.0/draft-ietf-http-spec.html
HTTP 1.1 - http://www.w3.org/Protocols/rfc2616/rfc2616.html
XML 1.0 - http://www.w3.org/TR/REC-xml
XML Namespaces 1.0 - http://www.w3.org/TR/1999/REC-xml-names-19990114/
XML Schema 1.0 - http://www.w3.org/TR/xmlschema-0/
WSDL 1.1 - http://www.w3.org/TR/wsdl
SOAP 1.1 - http://www.w3.org/TR/soap/
SOAP 1.2 - http://www.w3.org/2000/xp/Group/
SOAP with Attachments – http://www.w3.org/TR/SOAP-attachments
MTOM – http://www.w3.org/TR/soap12-mtom/
WS-I Basic Profile - http://www.ws-i.org/Profiles/BasicProfile-1.0-2004-04-16.html
WS-Addressing 1.0 - http://www.w3.org/Submission/ws-addressing/
WS-ReliableMessaging - http://schemas.xmlsoap.org/ws/2005/02/rm/
WS-Security 1.0 - http://www.ibm.com/developerworks/library/specification/ws-secure/
BPEL 1.1 - http://www.ibm.com/developerworks/library/specification/ws-bpel/
BPEL 2.0 - http://www.oasis-open.org/committees/download.php/10347/wsbpel-specification-draft-120204.htm (only partial support)
UDDI v3 - http://uddi.org/pubs/uddi_v3.htm
XPATH 1.0 - http://www.w3.org/TR/xpath
XSLT 1.0 - http://www.w3.org/TR/xslt
XQuery 1.0 - http://www.w3.org/TR/xquery/

Back to Top

FAULT RAISING AND HANDLING

In service-oriented messaging infrastructure, faults or exceptions can occur either at transport level or message processing level (such as SOAP) or business service level.

It is important to handle every fault scenario to build robust and predictable services while simultaneously providing visibility into the state of the system.

Broadly, exceptions or faults can be classified horizontally into the following categories:

- **Unrecoverable Exceptions:** Unrecoverable exceptions are unexpected fatal error conditions within a system that cannot be handled gracefully by the involved systems or services automatically and require manual handling.
- **Recoverable Exceptions:** Recoverable exceptions denote that some condition in the system has not been met for a given context but may be recoverable gracefully through alternative means by the involved systems or services automatically.

Again, broadly, exceptions can further be classified vertically into the following categories:

- **System Exceptions:** System exceptions are hardware or software related issues that are related to IT organization of an enterprise. Some of the system exceptions are recoverable and most of them are unrecoverable automatically and requires manual handling.
- **Business Exceptions:** Business exceptions are part of the business service interface and implementation contract having business semantics. These exceptions are expected by the involved parties (i.e., service producers and service consumers) and hence are recoverable through an alternative business process flow or human workflow.

Following are the system exception scenarios that can occur in a service-oriented system and the corresponding processing patterns:

- **System Resource Exceptions:** System resource exceptions indicate a fatal situation related to system resources. System Crash, Resource Unavailability, Out-of-Memory exception, etc. could fall under this category. These types of exceptions can either be handled manually or automatically. By acquiring an alternative resource, systems can automatically recover from these exceptions. For example, in case of resource unavailability, fail over to an alternative resource or retry mechanisms might allow the system to recover. But typically, resource exceptions are handled manually by restarting or reallocating resources.

  In case of unrecoverable exceptions, such as System Crash or Out-of-Memory exceptions, ‘networking and system management’ tools must be used to detect the situation and raise an alert to the involved parties. System Resource Exceptions must be raised and alert both business users as well as IT support personnel so that all the involved parties are aware of the system as well as business state. But System Resource Exceptions should be handled only by the IT support personnel.

  Synchronous MEP based Services will throw system resource exceptions back to the consumer at the time of consuming the service as a fault message.
- **Internal System Exceptions:** Internal system exceptions occur usually due to programming logic errors on the service provider side such as null pointer exceptions. Usually these exceptions occur when the provider is not cooperating to handle that particular interaction from the consumer’s perspective. Ideally these exception conditions should never occur. But practically it is impossible to avoid these exceptions.

  The service consumers cannot handle exceptions on the provider side and hence optionally exceptions may be raised to the consumer for visibility purposes, but the exception must be handled by the provider only. Exceptions must be logged and the provider must raise an appropriate alert for visibility purposes. Since it is an internal problem, the same message can be processed again after fixing the problem either automatically or manually. Internal System Exceptions must be raised and alert both business users as well as IT support personnel so that all the involved parties are aware of the system as well as business state. Internal System Exceptions should either be handled by the IT support personnel or business users or both depending upon the situation.

  Synchronous MEP based Services will throw internal system exceptions back to the consumer as a fault message.
- **External System Exceptions:** External system exceptions occur usually due to programming logic errors on the consumer side. Usually these exceptions occur when the consumer violates the interaction agreement from the provider’s perspective. Ideally these exception conditions should never occur. But practically it is impossible to avoid these kinds of exceptions.

  Exceptions due to consumer error should be thrown to the consumer by the provider so that it may be handled by the consumer by taking alternative actions and if

not possible, exceptions must be logged and an appropriate alert must be raised for visibility purposes by the consumer. External System Exceptions must be raised and alert both business users as well as IT support personnel so that all the involved parties are aware of the system as well as business state. External System Exceptions should either be handled by the IT support personnel or business users or both depending upon the business requirements.

Synchronous MEP based Services will throw external system exceptions back to the consumer as a fault message.

The details of raising and handling faults at every service-oriented messaging layer must be defined and an example is provided below:

Transport Layer

The fault bindings for the transports used from the consumer's perspective is defined below:

- HTTP
- System Resource Exception
  - 404 Not Found
  - 503 Service Unavailable
- Internal and External System Exception
  - 500 Internal Server Error

SOAP Layer

Exceptions are propagated to the service consumer using the SOAP1.1 Fault message. A SOAP1.1 fault message is an envelope that has a <soap:Fault> element as single child element of the <soap:Body> element and the <soap:Fault> element must not have element children other than <faultcode>, <faultstring>, <faultactor> and <detail>.

In case of HTTP transport binding, '500 Internal Server Error' status code must be used to carry the soap fault message.

Business Service Layer

Business faults must be propagated to the business service consumer as a literal document in a SOAP1.1 envelope.

Synchronous MEP based services will raise all the faults to the service consumer as SOAP Faults. The <detail> element of the SOAP Fault will carry the detailed Fault message.

From the BPEL perspective, throw and catch activities and fault handlers must be used appropriately. Also BPEL based human workflow process can be used to handle exceptions manually.

Fault handling in long running transaction scenarios might involve compensation handlers to bring the system back to a consistent state.

Back to Top

WS-I COMPLIAN WSDL TEMPLATES

The Web Services Interoperability Organization is an open industry effort chartered to promote Web services interoperability across platforms, applications, and programming languages. The organization brings together a diverse community of Web services leaders to respond to customer needs by providing guidance, recommended practices, and supporting resources for developing interoperable Web services.

Section 5 of the WS-I Basic Profile 1.0 specification relates to conformant use of WSDL to describe a Web service. Of all the sections in the profile specification, it is probably of most interest and importance to Web services developers. It begins by addressing issues related to WSDL document structure and composition.

The first issue addressed is that of correct usage of the wsdl:import feature. The wsdl:import feature is intended to be used in the same manner as the xsd:import feature, to import definitions from a foreign namespace. e.g. a namespace that is not the same as the value of the targetNamespace attribute of the importing WSDL document.

Additionally, the WS-I Basic Profile constrains use of the <wsdl:import> feature to the import of WSDL definition components from another WSDL document. This means that you cannot use the <wsdl:import> feature to import schema definitions from an XSD file.

The profile also constrains the order in which the top-level WSDL elements are to be used. Specifically, it requires that the wsdl:import element, if present, precede all other top-level WSDL elements. It is then followed by the <wsdl:types> element, and then <wsdl:message>, <wsdl:portType>, <wsdl:binding> and <wsdl:service> elements which may appear as many times as needed, and in any order.

This constraint, like many others in the specification, is based on decisions made by the W3C Working Group defining the next version of the Web services specifications for SOAP and WSDL. The reason that the Basic Profile WG has adopted certain decisions made by the W3C WGs is that they wanted to resolve identified interoperability issues in a manner consistent with the direction of the W3C WG drafting the 1.2 versions of the SOAP and WSDL specifications. This approach is intended to facilitate a smoother transition to the next version of the specifications once they reach W3C Recommendation status and are incorporated into subsequent WS-I Profiles.

SOAP Encoding Style

A WSDL binding describes how the service is bound to a messaging protocol, particularly the SOAP messaging protocol. WSDL 1.1 distinguishes between two soap message binding styles: document and RPC. Here's how each style affects the contents of <SOAP:Body>.

- **Document:** <SOAP:Body> contains one or more child elements called parts. There are no SOAP formatting rules for what the <SOAP:Body> contains; it contains whatever the sender and the receiver agree upon.
- **RPC:** RPC implies that <SOAP:Body> contains an element with the name of the method or remote procedure being invoked. This element in turn contains an element for each parameter of that procedure.

For applications that use serialization/deserialization to abstract away the data wire format, there's one more choice to be made: the serialization format. There are two popular serialization formats today:

- **SOAP Encoding:** SOAP encoding is a set of serialization rules defined in section 5 of SOAP 1.1 and is sometimes referred to as "section 5 encoding." The rules specify how objects, structures, arrays, and object graphs should be serialized. Generally speaking, an application using SOAP encoding is focused on remote procedure calls and will likely use RPC message style.
- **Literal:** Data is serialized according to a schema. In practice, this schema is usually expressed using W3C XML Schema. Although there are no prescribed rules for serializing objects, structures, and graphs, etc., the service's schema (usually XSD) describes the application-level Infoset of each of the service's messages.

Thus it gives us 4 different choices of soap binding styles. In addition to these 4 styles, a 5th binding style, which is commonly called the “document/literal wrapped”, is also available to choose. WS-I Basic Profile 1.0 disallows encoded content due to the complexity and incomplete support for the spec by various toolkits; only Document/Literal and RPC/Literal are allowed.

Document/Literal is highly interoperable over other styles (hence recommended by WS-I):

- Messaging oriented as opposed to RPC
- Enables loosely-coupled service implementation instead of tightly coupled
- Content within the <SOAP:Body> can be easily validated against a XML Schema (XSD)
- .NET Platform uses Document/Literal style as its default

Whereas the RPC/Encoded style has the following weaknesses:

- Encourages tightly coupled service implementation
- The type encoding info (such as xsi:type) is an overhead, degrading throughput performance
- Message validation using XSD cannot be easily validated

In essence, there are two compelling reasons to use document-style messaging. One is to gain the independence it provides. Its strength lies in decoupling interfaces between services, such as in application-to-application and system-to-system interfaces, to the point that they can change completely independently of one another. The other is that document-style messaging puts the full power of XML for structuring and encoding information at our disposal.

The following links provide the BPEL friendly WSDL templates (with Partner Link Types) corresponding to the message exchange patterns defined elsewhere in this document. These documents use specific naming conventions and can be reused by modifying the version, verb and noun strings appropriately.

- Sample 1 - SYNCHRONOUS-QUERY-RESPONSE-FAULT OR SYNCHRONOUS-REQUEST-RESPONSE-FAULT – MESSAGE EXCHANGE PATTERNS
- Sample 2 - SYNCHRONOUSE-REQUEST-ACK-FAULT – MESSAGE EXCHANGE PATTERN
- Sample 3 - ASYNCHRONOUS-ONE-WAY – MESSAGE EXCHANGE PATTERN
- Sample 4 - ASYNCHRONOUS-REQUEST-RESPONSE – MESSAGE EXCHANGE PATTERN
- Sample 5 - ASYNCHRONOUS-REQUEST-ACK OR ASYNCHRONOUS-REQUEST-ACK-WITH-TIMEOUT OR ASYNCHRONOUS-REQUEST-ACK-WITH-TIMEOUT-NOTIFICATION OR ASYNCHRONOUS-REQUEST-WITH-OPTIONAL-ACK-WITH-TIMEOUT – MESSAGE EXCHANGE PATTERN
- Sample 6 - ASYNCHRONOUS-REQUEST-RESPONSE-WITH-OPTIONAL-ACK-WITH-TIMEOUT – MESSAGE EXCHANGE PATTERNS

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Synchronous-Query-Response-Fault or Synchronous-Request-Response-Fault – Message Exchange Pattern

```
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounRequestMessage.xsd"/>
<xs:include schemaLocation="VerbNounResponseMessage.xsd"/>
<xs:include schemaLocation="VerbNounFaultMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounRequestMessage">
<wsdl:part name="VerbNounRequestPart" element="nvmm:VerbNounRequestMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounResponseMessage">
<wsdl:part name="VerbNounResponsePart" element="nvmm:VerbNounResponseMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounFaultMessage">
<wsdl:part name="VerbNounFaultPart" element="nvmm:VerbNounFaultMessage"/>
</wsdl:message>

<wsdl:portType name="VerbNounServicePT">
<wsdl:operation name="VerbNounOP">
```

```
<wsdl:input name="VerbNounRequestMessage" message="nvws:VerbNounRequestMessage"/>
<wsdl:output name="VerbNounResponseMessage" message="nvws:VerbNounResponseMessage"/>
<wsdl:fault name="VerbNounFaultMessage" message="nvws:VerbNounFaultMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceBinding" type="nvws:VerbNounServicePT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounOP">
<soap:operation style="document" soapAction="VerbNounOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
<wsdl:output>
<soap:body use="literal"/>
</wsdl:output>
<wsdl:fault name="VerbNounFaultMessage">
<soap:fault name="VerbNounFaultMessage" use="literal"/>
</wsdl:fault>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServicePT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions>
```

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Synchronous-Request-ACK-Fault – Message Exchange Pattern

```
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounRequestMessage.xsd"/>
<xs:include schemaLocation="VerbNounAcknowledgementMessage.xsd"/>
<xs:include schemaLocation="VerbNounFaultMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounRequestMessage">
<wsdl:part name="VerbNounRequestPart" element="nvmm:VerbNounRequestMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounAcknowledgementMessage">
<wsdl:part name="VerbNounAcknowledgementPart" element="nvmm:VerbNounAcknowledgementMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounFaultMessage">
<wsdl:part name="VerbNounFaultPart" element="nvmm:VerbNounFaultMessage"/>
</wsdl:message>

<wsdl:portType name="VerbNounServicePT">
<wsdl:operation name="VerbNounOP">
```

```
<wsdl:input name="VerbNounRequestMessage" message="nvws:VerbNounRequestMessage"/>
<wsdl:output name="VerbNounAcknowledgementMessage" message="nvws:VerbNounAcknowledgementMessage"/>
<wsdl:fault name="VerbNounFaultMessage" message="nvws:VerbNounFaultMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceBinding" type="nvws:VerbNounServicePT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounOP">
<soap:operation style="document" soapAction="VerbNounOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
<wsdl:output>
<soap:body use="literal"/>
</wsdl:output>
<wsdl:fault name="VerbNounFaultMessage">
<soap:fault name="VerbNounFaultMessage" use="literal"/>
</wsdl:fault>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServicePT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions>
```

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Asynchronous-One-Way – Message Exchange Pattern

```
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounMessage">
<wsdl:part name="VerbNounPart" element="nvmm:VerbNounMessage"/>
</wsdl:message>

<wsdl:portType name="VerbNounServiceProviderPT">
<wsdl:operation name="VerbNounOP">
<wsdl:input name="VerbNounMessage" message="nvws:VerbNounMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceProviderBinding" type="nvws:VerbNounServiceProviderPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounOP">
<soap:operation style="document" soapAction="VerbNounOP"/>
```

```
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServiceProviderPT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions
```

Back to Top

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Asynchronous-Request-Response – Message Exchange Pattern

```
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounRequestMessage.xsd"/>
<xs:include schemaLocation="VerbNounResponseMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounRequestMessage">
<wsdl:part name="VerbNounRequestPart" element="nvmm:VerbNounRequestMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounResponseMessage">
<wsdl:part name="VerbNounResponsePart" element="nvmm:VerbNounResponseMessage"/>
</wsdl:message>

<wsdl:portType name="VerbNounServiceProviderPT">
<wsdl:operation name="VerbNounRequestOP">
<wsdl:input name="VerbNounRequestMessage" message="nvws:VerbNounRequestMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:portType name="VerbNounServiceConsumerPT">
```

```
<wsdl:operation name="VerbNounResponseOP">
<wsdl:input name="VerbNounResponseMessage" message="nvws:VerbNounResponseMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceProviderBinding" type="nvws:VerbNounServiceProviderPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounRequestOP">
<soap:operation style="document" soapAction="VerbNounRequestOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<wsdl:binding name="VerbNounServiceConsumerBinding" type="nvws:VerbNounServiceConsumerPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounResponseOP">
<soap:operation style="document" soapAction="VerbNounResponseOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServiceProviderPT"/>
</plnk:role>
<plnk:role name="VerbNounServiceConsumer">
<plnk:portType name="nvws:VerbNounServiceConsumerPT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions>
```

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Asynchronous-Request-Ack or Asynchronous-Request-Ack-With-Timeout or Asynchronous-Request-Ack-With-Timeout-Notification or Asynchronous-Request-With-Optional-Ack-With-Timeout – Message Exchange Pattern


```xml
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounRequestMessage.xsd"/>
<xs:include schemaLocation="VerbNounAcknowledgementMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounRequestMessage">
<wsdl:part name="VerbNounRequestPart" element="nvmm:VerbNounRequestMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounAcknowledgementMessage">
<wsdl:part name="VerbNounAcknowledgementPart" element="nvmm:VerbNounAcknowledgementMessage"/>
</wsdl:message>
```

```
<wsdl:portType name="VerbNounServiceProviderPT">
<wsdl:operation name="VerbNounRequestOP">
<wsdl:input name="VerbNounRequestMessage" message="nvws:VerbNounRequestMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:portType name="VerbNounServiceConsumerPT">
<wsdl:operation name="VerbNounAcknowledgementOP">
<wsdl:input name="VerbNounAcknowledgementMessage" message="nvws:VerbNounAcknowledgementMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceProviderBinding" type="nvws:VerbNounServiceProviderPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounRequestOP">
<soap:operation style="document" soapAction="VerbNounRequestOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<wsdl:binding name="VerbNounServiceConsumerBinding" type="nvws:VerbNounServiceConsumerPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounAcknowledgementOP">
<soap:operation style="document" soapAction="VerbNounAcknowledgementOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServiceProviderPT"/>
</plnk:role>
<plnk:role name="VerbNounServiceConsumer">
<plnk:portType name="nvws:VerbNounServiceConsumerPT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions>
```

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

OUM Asynchronous-Request-Response-With-Optional-Ack-With-Timeout – Message Exchange Pattern

```
<?xml version="1.0" encoding="UTF-8"?>

<wsdl:definitions name="VerbNounService"
targetNamespace="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvws="http://services.xyz.com/YYYY/MM/DD/VerbNounService"
xmlns:nvmm="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:plnk="http://schemas.xmlsoap.org/ws/2003/05/partner-link/">
<wsdl:types>
<xs:schema attributeFormDefault="qualified"
elementFormDefault="qualified"
targetNamespace="http://schemas.xyz.com/YYYY/MM/DD"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="VerbNounRequestMessage.xsd"/>
<xs:include schemaLocation="VerbNounAcknowledgementMessage.xsd"/>
<xs:include schemaLocation="VerbNounResponseMessage.xsd"/>
</xs:schema>
</wsdl:types>

<wsdl:message name="VerbNounRequestMessage">
<wsdl:part name="VerbNounRequestPart" element="nvmm:VerbNounRequestMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounAcknowledgementMessage">
<wsdl:part name="VerbNounAcknowledgementPart" element="nvmm:VerbNounAcknowledgementMessage"/>
</wsdl:message>
<wsdl:message name="VerbNounResponseMessage">
<wsdl:part name="VerbNounResponsePart" element="nvmm:VerbNounResponseMessage"/>
</wsdl:message>

<wsdl:portType name="VerbNounServiceProviderPT">
<wsdl:operation name="VerbNounRequestOP">
```

```
<wsdl:input name="VerbNounRequestMessage" message="nvws:VerbNounRequestMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:portType name="VerbNounServiceConsumerPT">
<wsdl:operation name="VerbNounAcknowledgementOP">
<wsdl:input name="VerbNounAcknowledgementMessage" message="nvws:VerbNounAcknowledgementMessage"/>
</wsdl:operation>
<wsdl:operation name="VerbNounResponseOP">
<wsdl:input name="VerbNounResponseMessage" message="nvws:VerbNounResponseMessage"/>
</wsdl:operation>
</wsdl:portType>

<wsdl:binding name="VerbNounServiceProviderBinding" type="nvws:VerbNounServiceProviderPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounRequestOP">
<soap:operation style="document" soapAction="VerbNounRequestOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<wsdl:binding name="VerbNounServiceConsumerBinding" type="nvws:VerbNounServiceConsumerPT">
<soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
<wsdl:operation name="VerbNounAcknowledgementOP">
<soap:operation style="document" soapAction="VerbNounAcknowledgementOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
<wsdl:operation name="VerbNounResponseOP">
<soap:operation style="document" soapAction="VerbNounResponseOP"/>
<wsdl:input>
<soap:body use="literal"/>
</wsdl:input>
</wsdl:operation>
</wsdl:binding>

<plnk:partnerLinkType name="VerbNounServicePLT">
<plnk:role name="VerbNounServiceProvider">
<plnk:portType name="nvws:VerbNounServiceProviderPT"/>
</plnk:role>
<plnk:role name="VerbNounServiceConsumer">
<plnk:portType name="nvws:VerbNounServiceConsumerPT"/>
</plnk:role>
</plnk:partnerLinkType>

</wsdl:definitions>
```

Back to Top

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.